



IES Belén - Málaga

Técnico Superior en Desarrollo de
Aplicaciones Web (DAW)

Curso 2025/2026

Orbit Control

Sistema SaaS de Gestión Integral
para Heladerías Artesanales

"Mantén tu negocio en órbita"



Raúl Rodríguez Aponte

TFG 2025/2026



ÍNDICE

- 1. Título Del Proyecto**
- 2. Estudio Del Sector Productivo**
 - 2.1. El Sector Heladero Artesanal En España**
 - 2.2. Problemáticas Del Sector**
 - 2.3. Competencia y Oportunidad De Mercado**
- 3. Finalidad Del Proyecto**
- 4. Descripción Básica Del Proyecto**
- 5. Descripción Detallada Del Proyecto**
 - 5.1. Herramientas De Desarrollo Software Utilizadas**
 - 5.2. Diseño Del Proyecto**
 - 5.3. Esquema De Funcionamiento De La Aplicación**
 - 5.4. Descomposición Modular e Interrelación entre módulos**
 - 5.5. Descripción De Los Módulos Del Proyecto**
 - 5.6. Descripción De La Interfaz De Usuario**
 - 5.7. Descripción De Listados E Informes**
 - 5.8. Manual De Usuario**
 - 5.9. Manual De Instalación O Despliegue**
 - 5.10. Presentación De La Aplicación (Vídeo)**
- 6. Planificación Del Proyecto**
- 7. Control de la Ejecución**
- 8. Dificultades Encontradas**
- 9. Propuestas De Mejora**
- 10. Conclusión Final**

1.- Título Del Proyecto



Sistema SaaS de Gestión Integral para Heladerías Artesanales

Orbit Control es una aplicación web desarrollada como Trabajo de Fin de Grado del ciclo formativo de Técnico Superior en Desarrollo de Aplicaciones Web (DAW) en el IES Belén de Málaga, durante el curso 2025/2026.

El nombre del proyecto surge de la idea de tener el negocio siempre bajo control, como si todo girara en **órbita** alrededor de un mismo centro. Igual que los planetas mantienen su trayectoria de forma precisa y ordenada, Orbit Control busca que cada parte del negocio, el obrador, el almacén, los pedidos y los proveedores, funcione de forma **coordinada** y **sin fricciones**. El slogan del proyecto, **"Mantén tu negocio en órbita"**, resume exactamente esa idea.

El proyecto ha sido desarrollado por **Raúl Rodríguez Aponte**, estudiante del segundo curso del ciclo DAW en el IES Belén. Durante el desarrollo del proyecto he compaginado los estudios con mi trabajo como responsable del obrador en una heladería artesanal de Málaga, lo que me ha permitido conocer de primera mano las necesidades reales del sector y trasladarlas directamente al diseño y funcionalidad de la aplicación.

Esta experiencia profesional ha sido clave para que Orbit Control no sea simplemente un proyecto académico, sino una herramienta con vocación real de llegar al mercado.

2.- Estudio Del Sector Productivo

El sector de las pequeñas y medianas empresas del ámbito alimentario artesanal representa uno de los pilares de la economía local española.

Negocios como heladerías, panaderías, pastelerías y obradores de producción comparten una característica común: una **gestión operativa compleja** que combina producción, inventario, proveedores y personal, pero que en la mayoría de los casos sigue haciéndose mediante **métodos manuales** o con herramientas genéricas **no adaptadas** a sus necesidades reales.

La transformación digital de este sector es todavía escasa, lo que genera una demanda creciente de soluciones software, asequibles y fáciles de usar.

2.1.- El Sector Heladero Artesanal En España

España cuenta con entre 300 y 400 fábricas de helado artesanal, principalmente en comunidades mediterráneas como Andalucía, Valencia y Murcia, aunque las hay por todo el país.

Al ser un sector muy estacional, con la mayor parte de la producción concentrada entre marzo y octubre, gestionar bien el inventario, los proveedores y el personal durante esos meses es clave para que el negocio funcione y sea rentable.

Además, muchas de estas empresas cuentan con varias tiendas propias o puntos de venta franquiciados, lo que añade aún más complejidad a su gestión diaria. Coordinar la producción del obrador con la demanda de cada tienda es uno de los mayores retos operativos del sector.

2.2.- Problemáticas Del Sector

La mayoría de las heladerías artesanales siguen gestionando su negocio con **hojas de cálculo, cuadernos en papel o aplicaciones genéricas** que no están pensadas para este sector. Esto provoca **errores frecuentes, pérdida de información** y una **falta de visibilidad total** sobre lo que ocurre en el negocio.

Uno de los problemas más habituales es la falta de trazabilidad en el obrador. No existe un registro digital claro de qué ingredientes se han consumido en cada producción, lo que dificulta el control de costes y la gestión del stock.

Los pedidos a proveedores se gestionan mayoritariamente por teléfono o WhatsApp, sin ningún tipo de seguimiento. Es común que se produzcan **roturas de stock** en plena temporada por no tener un control de las compras.

2.3.- Competencia y Oportunidad De Mercado

El mercado del software de gestión para heladerías artesanales está **prácticamente vacío**. No existe actualmente ninguna solución diseñada específicamente para este sector en España. Las alternativas que más se le acercan son herramientas genéricas como *Holded* u *Odoo*, que requieren una **configuración compleja y costosa** para adaptarlas a las necesidades de una heladería.

Los TPV tradicionales cubren **únicamente** el punto de venta, dejando fuera todo lo relacionado con la producción, el obrador o la gestión de proveedores. Son herramientas parciales que **no** resuelven el problema real.

Esto supone una **oportunidad clara** para una solución vertical como **Orbit Control**, que nace directamente desde el conocimiento del sector y está diseñada desde cero para cubrir las **necesidades reales**.

3.- Finalidad Del Proyecto

La **finalidad principal** de *Orbit Control* es ofrecer a las heladerías artesanales una herramienta de gestión **completa, sencilla y asequible**, que les permita **digitalizar y centralizar** todos sus procesos operativos en un **único lugar**. El objetivo es que cualquier heladería, independientemente de su tamaño, pueda acceder a un software pensado específicamente para su sector sin necesidad de grandes inversiones ni conocimientos técnicos.

Uno de los pilares del proyecto es el **control del obrador**. Orbit Control permite registrar cada producción de helado, calcular automáticamente los ingredientes consumidos según la receta y actualizar el stock del almacén en tiempo real. Esto **elimina** el trabajo manual y **reduce** los errores habituales en la gestión diaria.

Otro objetivo clave es mejorar la **gestión del inventario**. La aplicación avisa cuando un ingrediente está por debajo del mínimo configurado, **facilitando** la toma de decisiones antes de que se produzca una rotura de stock.

En cuanto a la **relación con proveedores**, Orbit Control centraliza todo el ciclo de compras, desde la creación del pedido hasta su recepción, **integrando automáticamente** las entradas de stock al confirmar la llegada de la mercancía.

4.- Descripción Básica Del Proyecto

Orbit Control es una aplicación web de gestión diseñada específicamente para heladerías artesanales. Funciona como un **SaaS**, es decir, el cliente no necesita instalar nada, simplemente accede desde el navegador de cualquier dispositivo, ya sea un ordenador, una tablet o un móvil.

La aplicación está dividida en módulos, cada uno enfocado en un área concreta del negocio: el almacén, el obrador, los pedidos, los proveedores y la gestión de usuarios.

Todos están conectados entre sí, de forma que una acción en un módulo **repercute automáticamente** en los demás. Por ejemplo, al registrar una producción en el obrador, el stock del almacén se actualiza solo.

Uno de los aspectos más destacados es su **sistema de autenticación**. Además del acceso tradicional con usuario y contraseña, Orbit Control permite entrar mediante **PIN numérico** o **tarjeta NFC**, algo especialmente útil para los empleados del obrador que trabajan con las manos ocupadas o sin acceso a un teclado.

Técnicamente, el **frontend** está desarrollado con **Angular 21** y el **backend** con **Spring Boot 3**, usando **JWT** para la **autenticación** y **MySQL** como **base de datos**.

La aplicación está empaquetada como **PWA**, lo que permite instalarla en cualquier dispositivo como si fuera una app nativa, sin pasar por ninguna tienda de aplicaciones.

5.- Descripción Detallada Del Proyecto

En este apartado se describe en detalle el proceso de desarrollo de Orbit Control, desde las herramientas utilizadas hasta el diseño de la base de datos, la arquitectura de la aplicación y cada uno de sus módulos.

El objetivo es dar una visión completa y técnica del proyecto, explicando no solo el qué sino también el porqué de cada decisión tomada durante el desarrollo.

5.1.- Herramientas De Desarrollo Software Utilizadas

Antes de entrar en detalle por cada área, es importante entender que la elección del stack tecnológico no fue aleatoria.

Cada herramienta se seleccionó buscando un equilibrio entre solidez técnica, curva de aprendizaje y adecuación al tipo de proyecto.

Se priorizaron tecnologías modernas, con soporte activo y amplia documentación, que permitieran construir una aplicación escalable y mantenible a largo plazo.

5.1.1.- Frontend

El frontend de Orbit Control está desarrollado con **Angular 21**, un framework mantenido por Google orientado a la construcción de aplicaciones web complejas.

Angular fue la opción natural ya que es el framework estudiado durante el módulo de **Desarrollo Web en Entorno Cliente**, lo que permitió familiarizarse rápidamente con el funcionamiento y centrarse desde el principio en adaptarlo a las necesidades concretas del proyecto.

Una de las características más aprovechadas de **Angular** ha sido su sistema de **componentes reutilizables**. A lo largo del desarrollo se han creado componentes compartidos como **tablas, formularios y modales** que se replican en los distintos módulos de la aplicación, evitando **duplicar código** y manteniendo una **interfaz visual consistente** en toda la aplicación.

Para los estilos se ha utilizado **Tailwind CSS**, un framework que permite construir interfaces personalizadas aplicando clases directamente en el HTML. Una de sus ventajas más aprovechadas en el proyecto ha sido su sistema **responsive**, que permite adaptar la interfaz a cualquier tamaño de pantalla sin necesidad de escribir media queries personalizadas, algo especialmente útil dado que Orbit Control está pensada para usarse tanto en el obrador desde una tablet como en una oficina desde un ordenador.

5.1.2.- Backend

El backend está construido con **Spring Boot 3** sobre **Java 21**, una de las combinaciones más sólidas y maduras para desarrollar **APIs REST**. Spring Boot simplifica enormemente la configuración del proyecto y permite centrarse en la lógica de negocio sin perder tiempo en configuraciones innecesarias.

La seguridad de la aplicación se gestiona con **Spring Security**, usando **JWT** como mecanismo de autenticación. Al ser un sistema sin estado, cada petición que llega al servidor incluye el **token** con la **información del usuario**, lo que elimina la necesidad de mantener sesiones y facilita la escalabilidad de la aplicación.

Para la comunicación con la base de datos se ha utilizado **Spring Data JPA** con **Hibernate** como implementación **ORM**, lo que permite trabajar directamente con objetos Java en lugar de escribir queries SQL en la mayor parte del código, simplificando bastante el desarrollo de la capa de persistencia.

5.1.3.- Base De Datos

Para la base de datos se ha utilizado MySQL 8, un sistema gestor de bases de datos relacional ampliamente utilizado en entornos de producción. Se eligió por su estabilidad, su rendimiento en aplicaciones de gestión y su integración directa con Spring Boot a través de Spring Data JPA.

5.1.4.- Infraestructura Y Despliegue

Para el despliegue se ha configurado un **pipeline CI/CD automatizado** que arranca con un simple `git push origin main`, desplegando frontend y backend de forma simultánea y sin intervención manual.

El frontend está desplegado en **Vercel**, que tiene integración nativa con GitHub. Al detectar un nuevo commit en la rama `main`, lanza automáticamente el build de Angular y despliega el resultado.

El **backend** corre en un servidor privado de **Hetzner** (CX43). El despliegue se gestiona mediante **GitHub Actions**, que ejecuta tres pasos de forma automática: compila el **JAR** con *Maven*, lo sube al servidor mediante **SCP** y reinicia el servicio con `systemctl`. La autenticación **SSH** se gestiona mediante una **Secret KEY** almacenada en GitHub.

El proyecto cuenta con dos entornos diferenciados: **desarrollo** (`dev.orbitcontrol.es`) y **producción** (`app.orbitcontrol.es`), cada uno con su propia **API** backend (`api-dev.orbitcontrol.es` y `api.orbitcontrol.es`), lo que permite probar los cambios antes de llevarlos al entorno real.

El control de versiones se ha llevado a cabo con **Git**, usando **GitHub** como repositorio remoto, con ramas diferenciadas para el desarrollo y la documentación del TFG.



5.1.5.- Entorno de Desarrollo

Para el desarrollo del backend se ha utilizado **IntelliJ IDEA**, uno de los *IDEs* más completos para **Java** y **Spring**, con soporte nativo para **Maven**, depuración integrada y herramientas específicas para **Spring Boot** que agilizan bastante el desarrollo.

El frontend se ha desarrollado con **Visual Studio Code**, complementado con extensiones específicas para Angular y TypeScript. Para el testing de la API se ha usado **Postman**, que permite probar y organizar los endpoints de forma cómoda durante el desarrollo.

Para arrancar el proyecto en local se ha creado un script centralizado en el **package.json** raíz que levanta el backend y el frontend de forma simultánea con un único comando, lo que agiliza bastante el flujo de desarrollo del día a día. Los comandos concretos y su uso se detallan en el apartado [5.9 Manual de instalación y despliegue.](#)



5.1.6.- Versiones

<i>Herramienta</i>	<i>Versión</i>	<i>Uso</i>
BACKEND		
Java (OpenJDK)	21.0.10	Runtime del backend
Spring Boot	3.4.2	Framework principal
Spring Security + JWT	0.12.5	Seguridad y autenticación
MapStruct	1.6.3	Mapeo de DTOs
Firebase Admin SDK	9.3.0	Notificaciones push
FRONTEND		
Angular	21.1.0	Framework principal
TypeScript	5.9.2	Lenguaje del frontend
RxJS	7.8.0	Programación reactiva
Tailwind CSS	4.1.12	Estilos y diseño responsive
INFRAESTRUCTURA		
MySQL	8.0.45	Base de datos relacional
GitHub Actions	-	CI/CD backend
Vercel	-	CI/CD + hosting frontend
Hetzner CX43	Ubuntu 24.04	Servidor VPS

5.2.- Diseño Del Proyecto

El diseño de la base de datos de Orbit Control surgió de un análisis directo de los procesos reales de una heladería artesanal.

El objetivo era modelar **fielmente** cómo funciona el negocio: desde la gestión de ingredientes y proveedores hasta la elaboración de helados en el obrador, pasando por el control de pedidos y el acceso por roles.

A lo largo del desarrollo el modelo fue evolucionando y adaptándose a necesidades que no siempre estaban previstas desde el principio.

5.2.1.- Decisiones de diseño destacadas

Relación polimórfica en lotes → La tabla **lotes** utiliza una relación **polimórfica** mediante los campos **tipo_entidad** y **entidad_id**.

En lugar de crear columnas separadas para cada tipo de entidad, lo que hubiera generado **columnas nulas** constantemente, se optó por una única columna genérica acompañada de su tipo.

Subrecetas → La tabla **ingredientes_receta** incluye el campo **id_subreceta**, que permite referenciar otra receta como si fuera un **ingrediente más**.

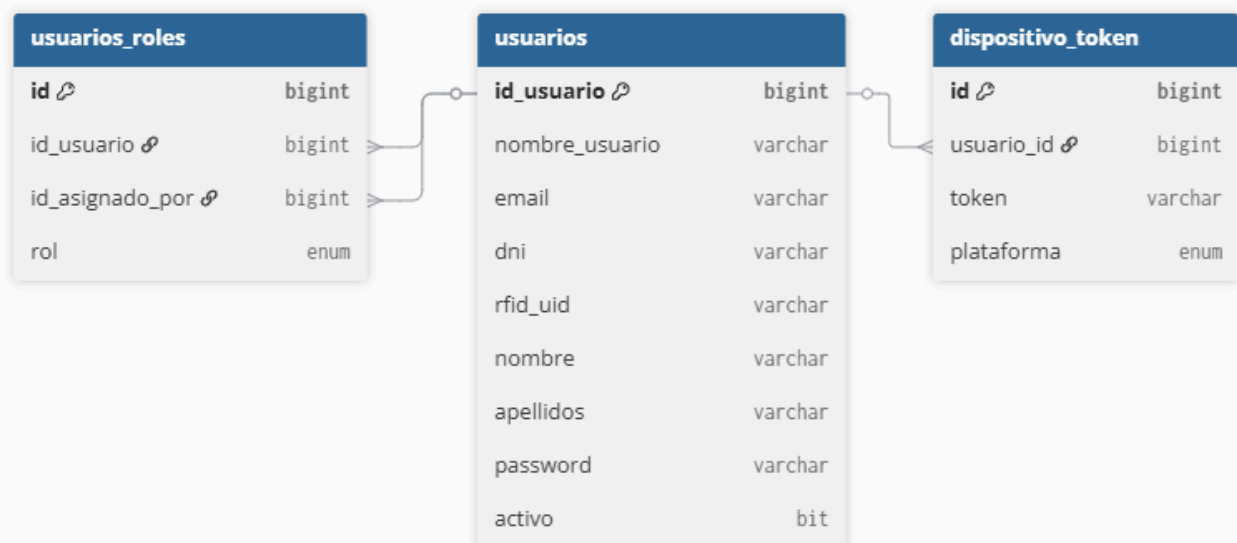
Esto responde a una necesidad real del sector: hay bases y mezclas intermedias que tienen su propia receta y que se usan como componente dentro de otras recetas más complejas. En lugar de duplicar ingredientes, se reutiliza directamente la receta base.

Separación entre helados y helados_elaborados → Se separaron en dos tablas distintas el helado como producto del catálogo y cada unidad física elaborada en el obrador. Esto permite tener **trazabilidad** completa de cada producción.

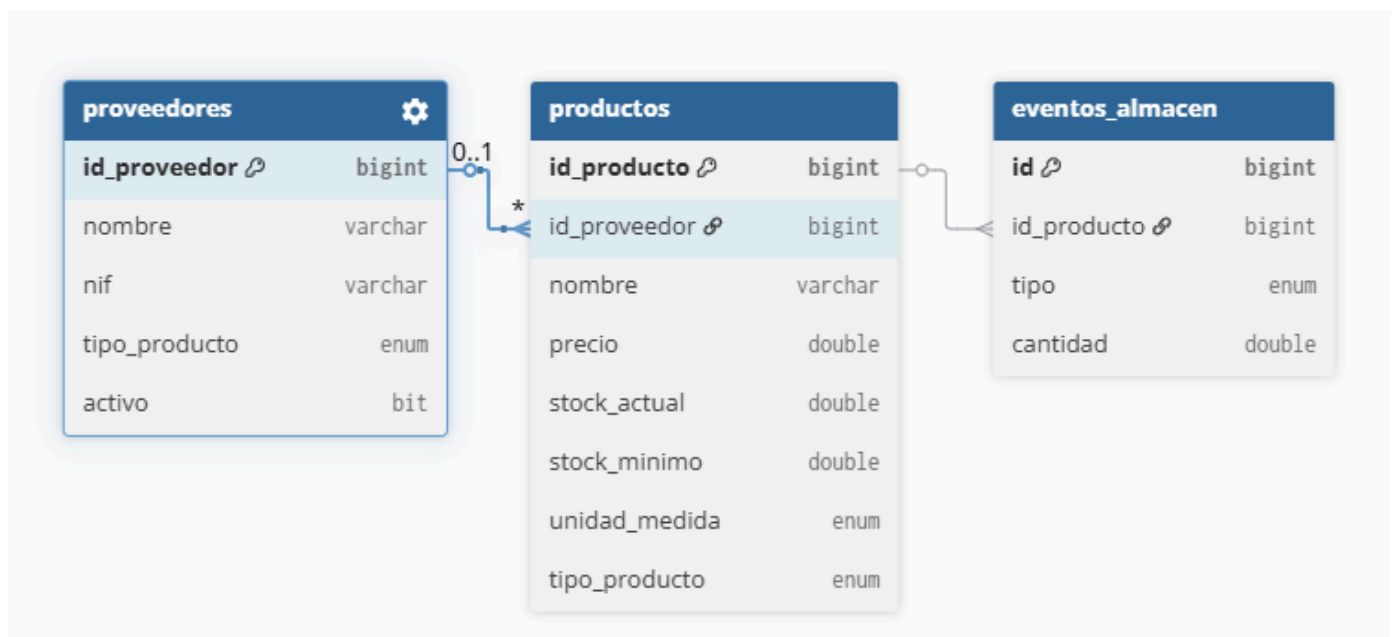
Roles diferenciados en el mismo nivel → Los roles **HELADERO** y **DEPENDIENTE** comparten el *nivel 3* pero tienen accesos orientados a áreas distintas del negocio. El **HELADERO** está enfocado al obrador y la elaboración, mientras que el **DEPENDIENTE** está orientado a la tienda y la consulta de stock. Esta separación permite ajustar los permisos a la realidad del trabajo diario sin necesidad de un sistema de permisos configurables más complejo.

5.2.2.- Modelo Entidad - Relación (E/R)

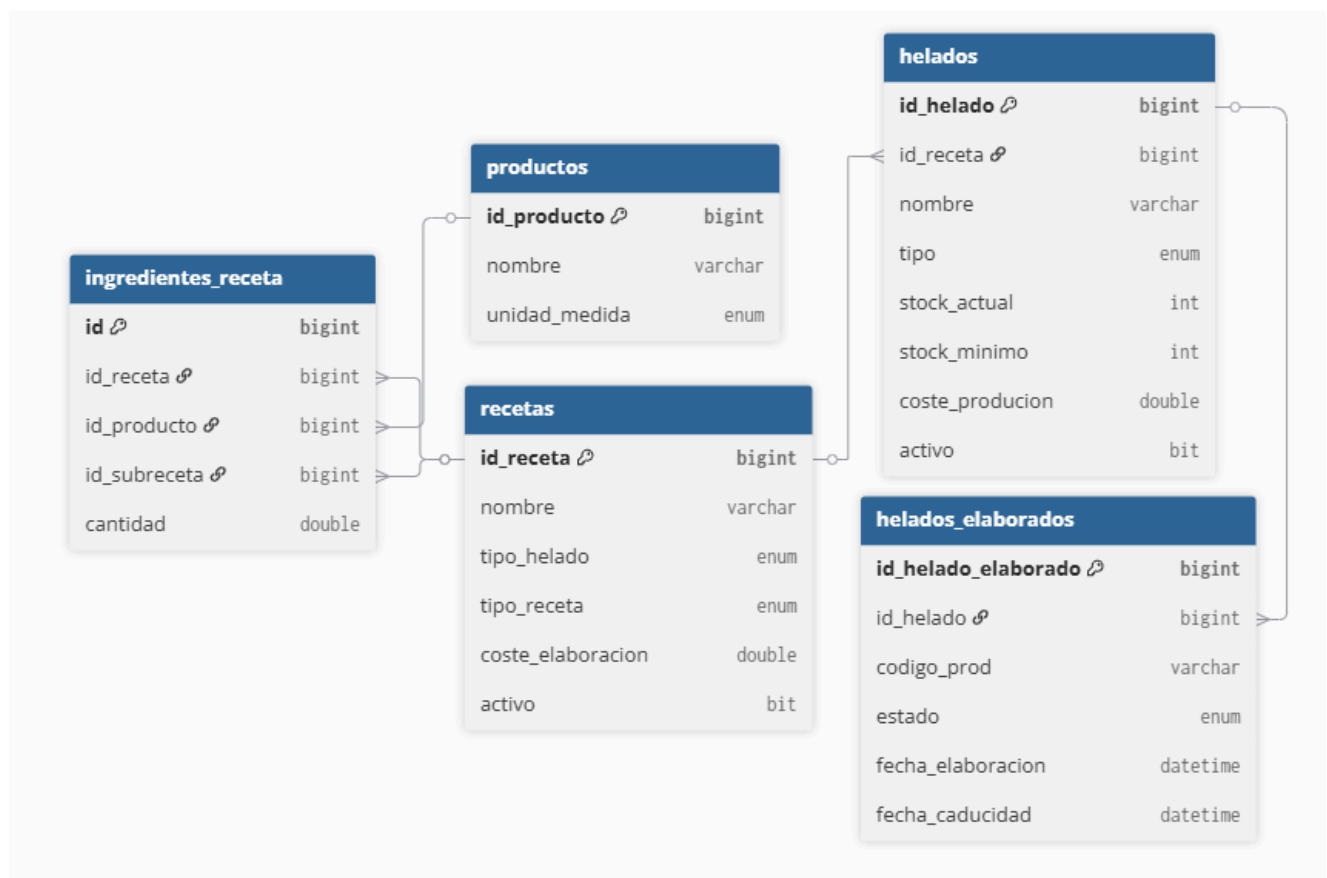
El modelo de datos de Orbit Control está compuesto por **15 tablas organizadas** en **cuatro bloques** funcionales. Para facilitar su lectura se presenta dividido por áreas, ya que un único diagrama con todas las relaciones resultaría difícil de interpretar en formato documento.



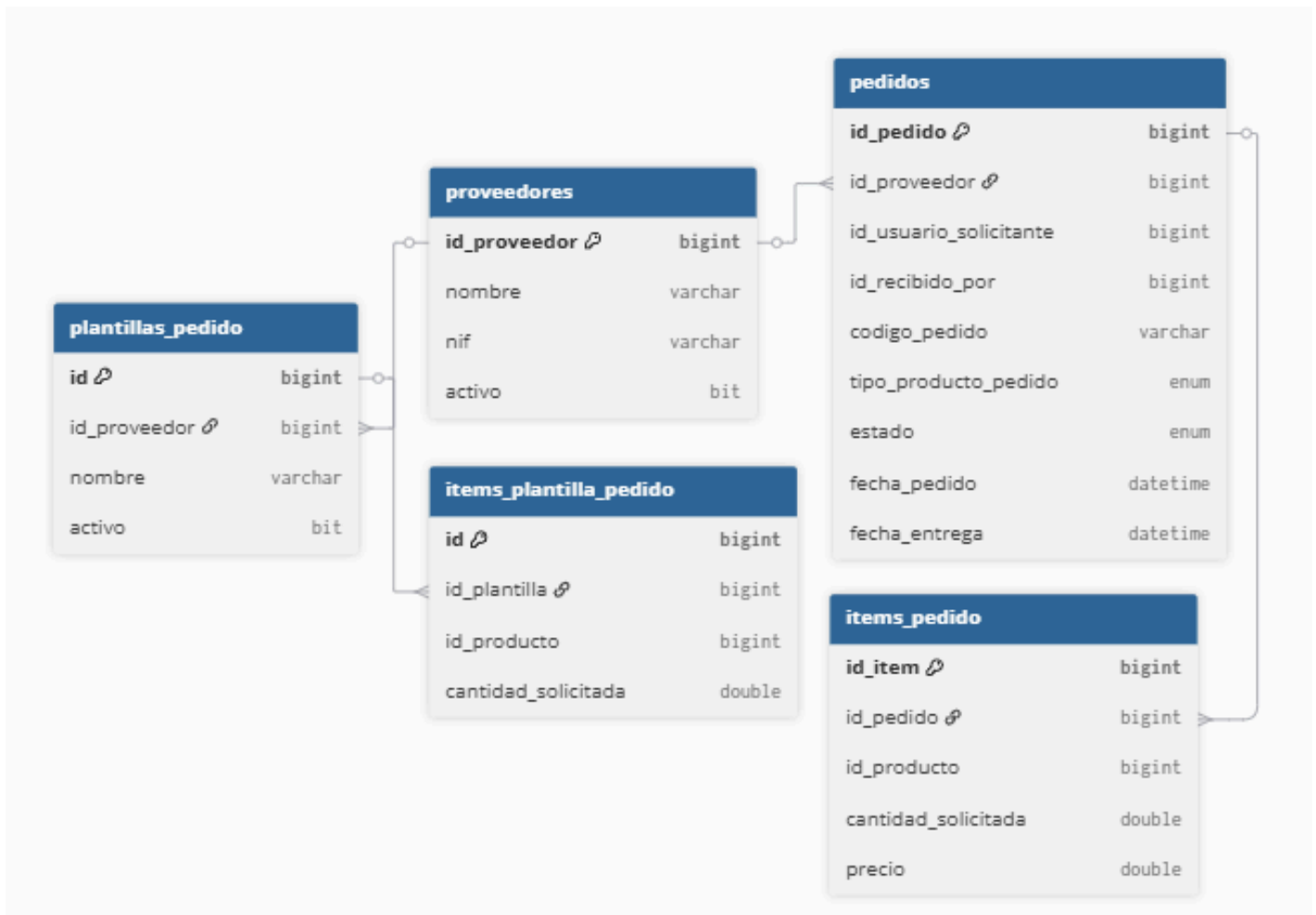
Bloque 1 - Usuarios y autenticación



Bloque 2 - Almacén y productos



Bloque 3 - Recetas y obrador



Bloque 4 - Pedidos y proveedores

5.2.3.- Diagramas De Casos De Uso

A continuación se representan los casos de uso de Orbit Control organizados por módulo. Cada diagrama muestra los actores que intervienen y las acciones que pueden realizar según su rol, diferenciando los flujos de éxito en verde y los casos de error en rojo.

Diagrama 1 - Autenticación

Todos los Usuario

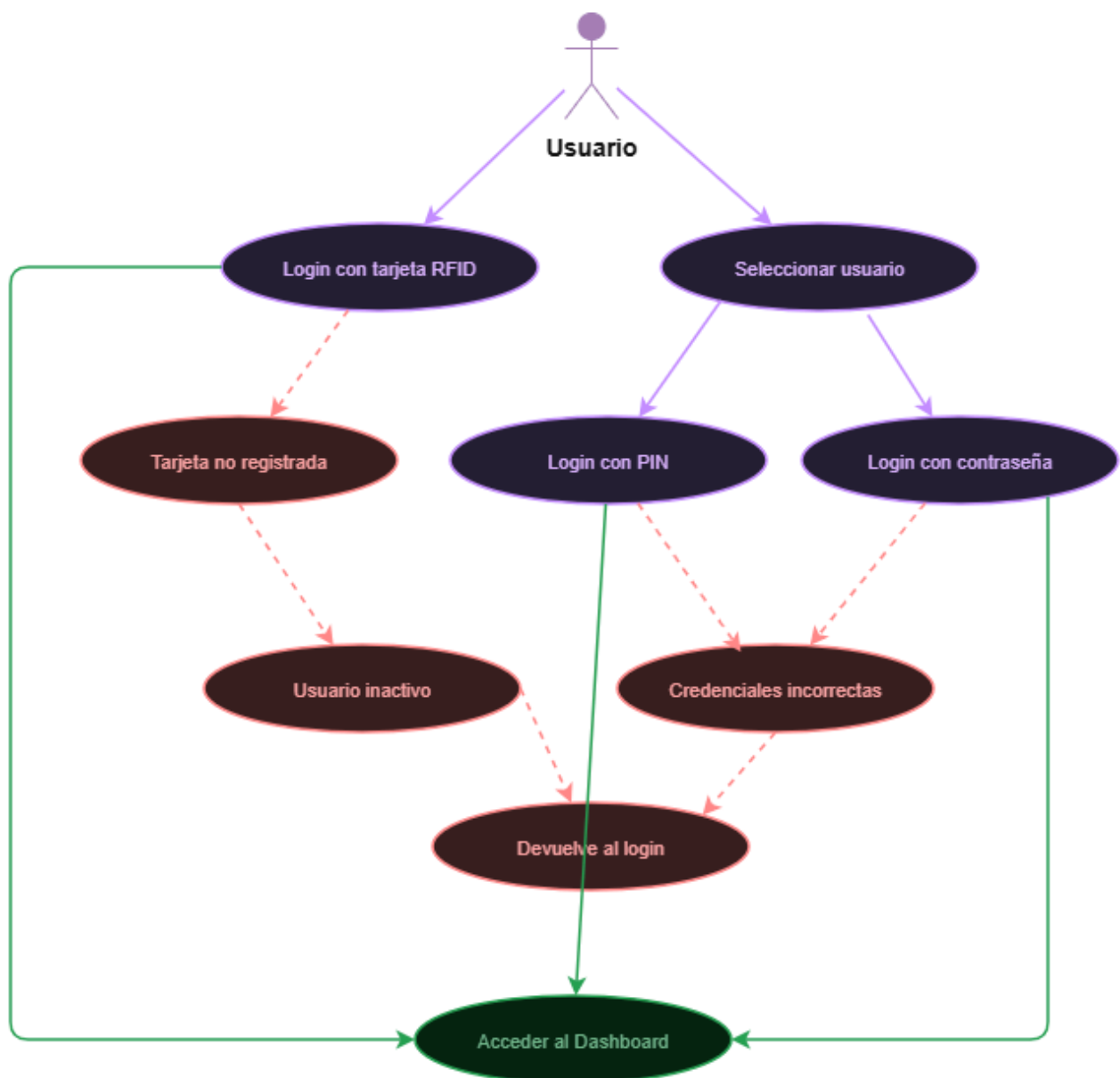


Diagrama 2 - Elaborar Helado

Heladero - Admin

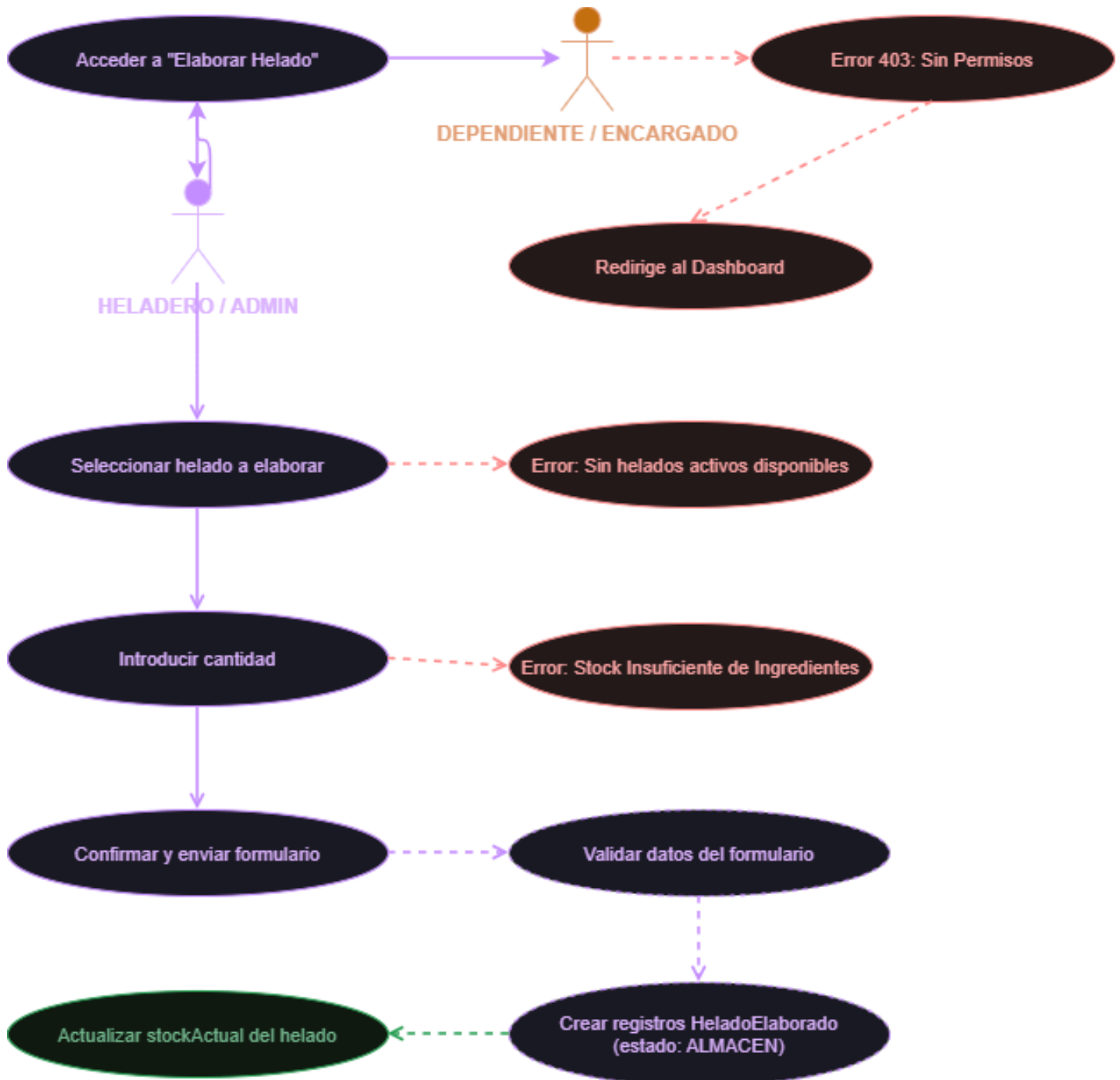


Diagrama 3 - Nuevo Pedido

Encargado - Admin



Diagrama 4 - Consumir Producto

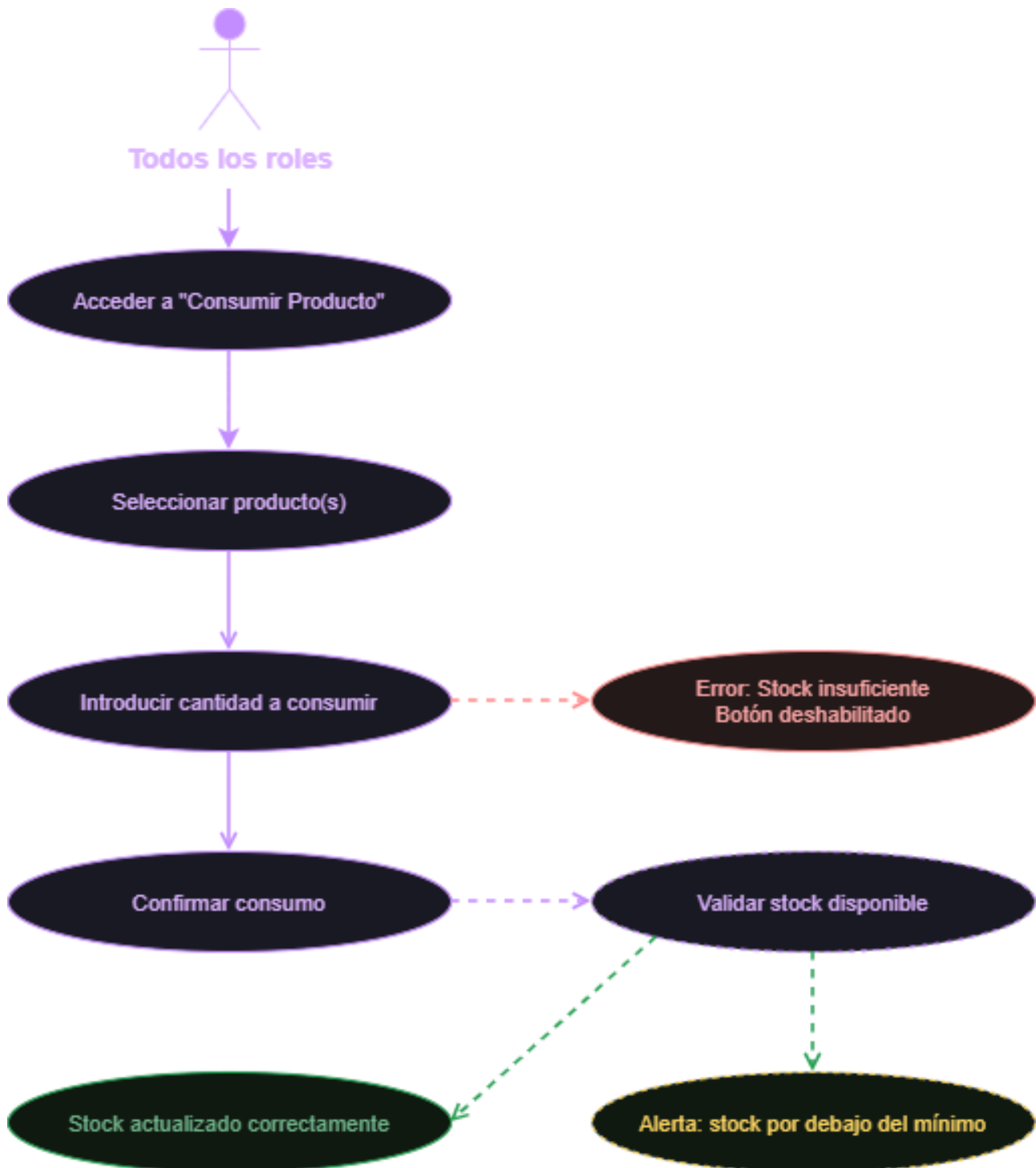
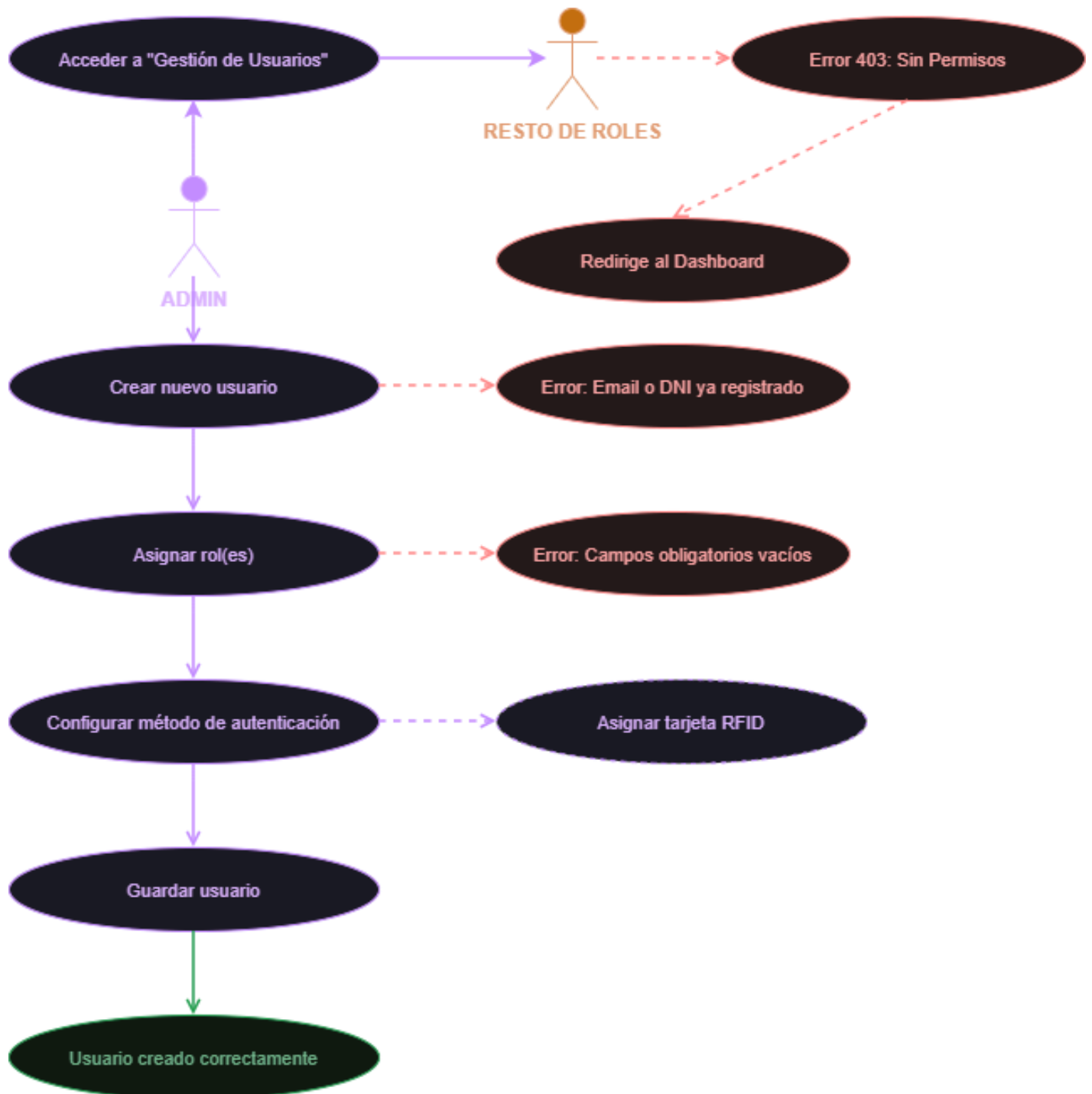
Todos los Usuarios

Diagrama 5 - Crear Usuario

Admin



5.2.4.- Diagrama De Arquitectura

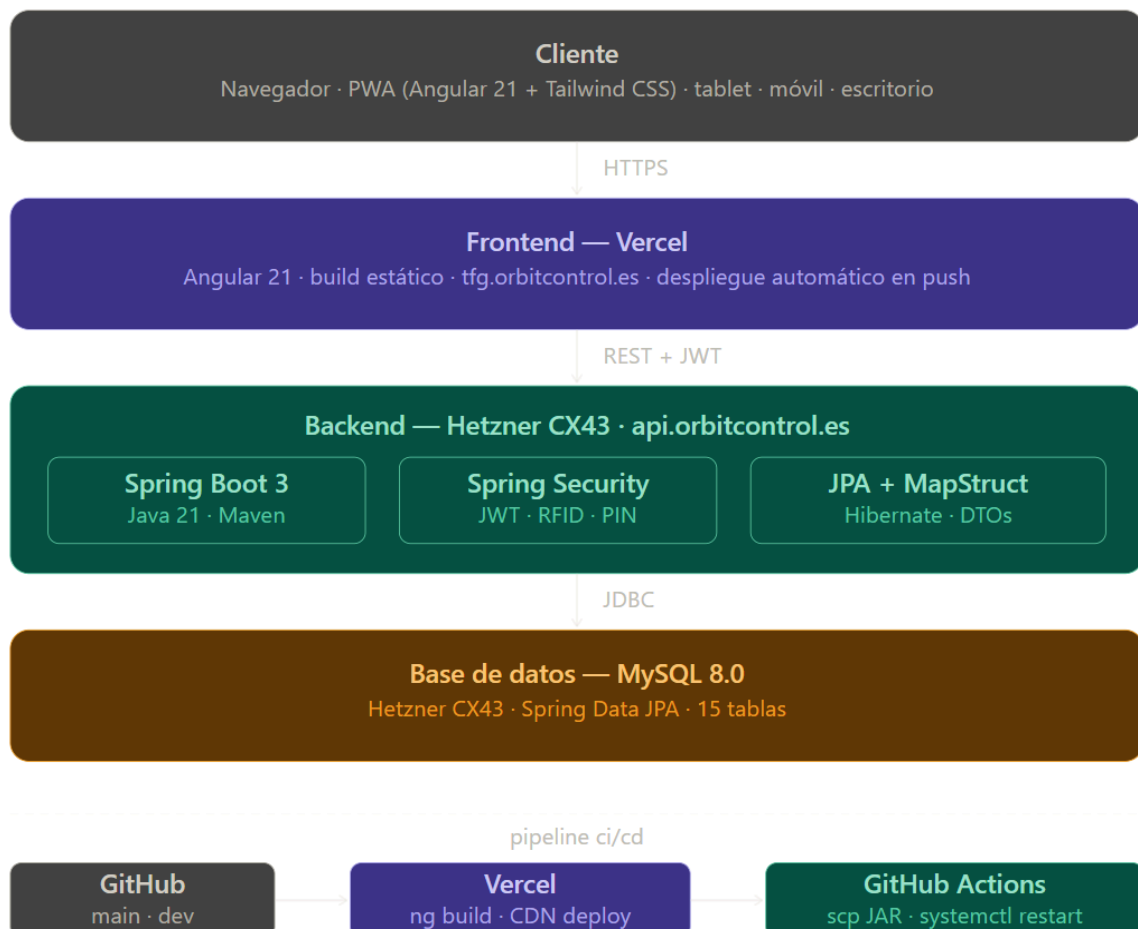
El siguiente diagrama muestra la arquitectura general de Orbit Control, organizada en **cuatro** capas principales.

En la parte superior se sitúa el **cliente**, que accede a la aplicación desde cualquier dispositivo a través del navegador o mediante la PWA instalada.

La capa de **frontend** está desplegada en *Vercel* y sirve el build estático de Angular 21 bajo el dominio tfg.orbitcontrol.es.

Las peticiones se comunican con el **backend** mediante una **API REST** con autenticación *JWT*, que corre en un servidor privado de *Hetzner CX43*. Por último, la capa de datos está compuesta por una base de datos MySQL 8.0 con 15 tablas, también alojada en el mismo servidor.

En la parte inferior del diagrama se representa el *pipeline CI/CD*, que automatiza el despliegue completo con un simple push a la rama correspondiente de GitHub.



5.2.5.- Diagrama De Despliegue

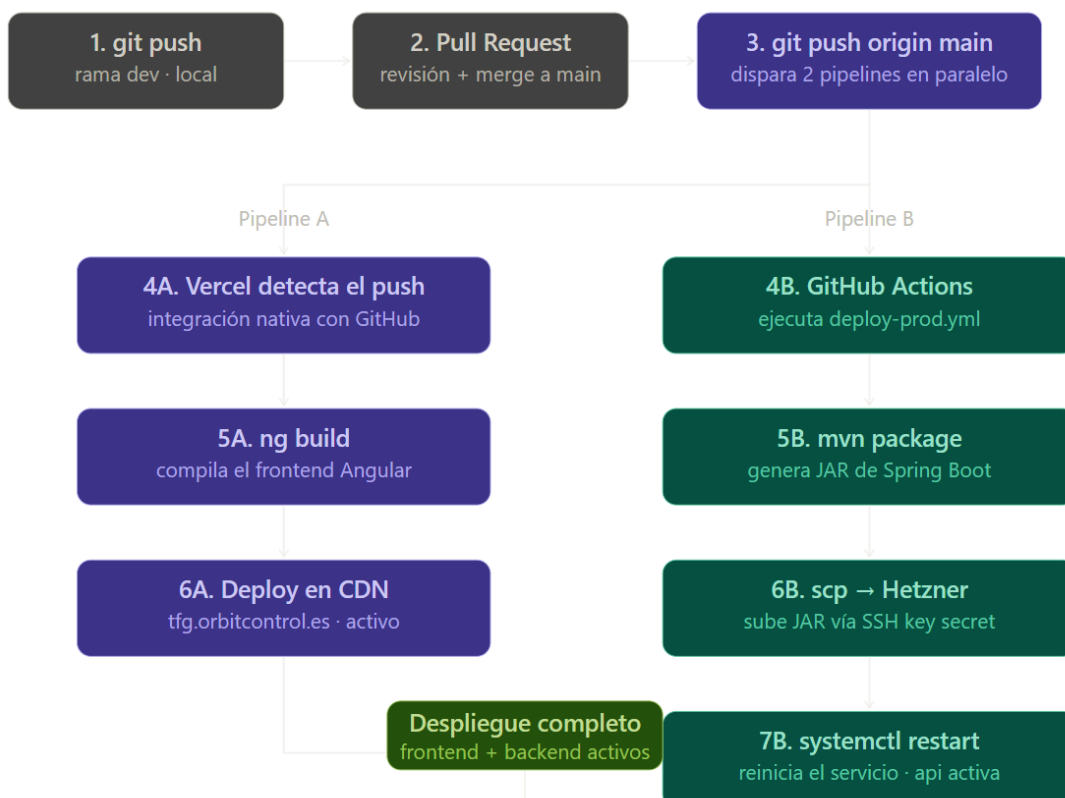
El despliegue está completamente **automatizado** mediante un **pipeline CI/CD**.

El proceso comienza con el desarrollo en la rama **dev**. Una vez que el código está listo para producción, se abre un **Pull Request** hacia la rama **main**, se revisa y se fusiona. En ese momento, un simple push a main dispara simultáneamente dos pipelines independientes.

Por un lado, **Vercel** detecta automáticamente el nuevo commit y lanza el build del frontend Angular, desplegando el resultado en su red de distribución global bajo el dominio tfg.orbitcontrol.es.

Por otro, **GitHub Actions** ejecuta el **workflow** de despliegue del backend: compila el **JAR** de Spring Boot con Maven, lo sube al servidor de **Hetzner** mediante **SCP** usando una clave **SSH** almacenada como secret en GitHub, y reinicia el servicio con **systemctl**.

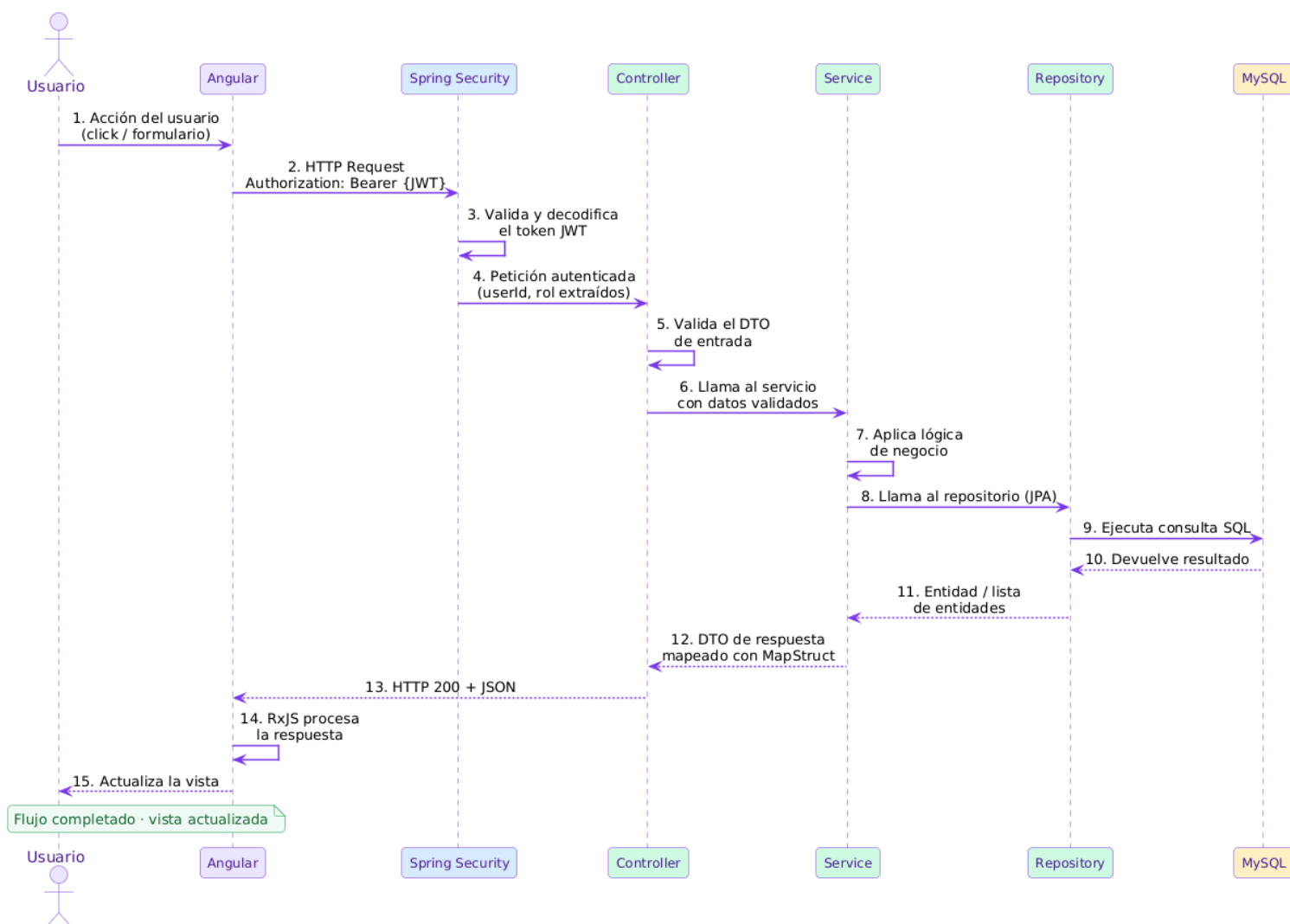
El resultado es que en pocos minutos tanto el frontend como el backend están actualizados y operativos sin haber tocado el servidor manualmente.



5.3.- Esquema De Funcionamiento De La Aplicación

Orbit Control sigue una arquitectura **cliente-servidor** donde el frontend y el backend se comunican exclusivamente a través de una **API REST**. El siguiente diagrama muestra el flujo completo de una petición, desde que el usuario realiza una acción en la interfaz hasta que la vista se actualiza con el resultado.

Cada petición que sale del frontend Angular incluye en su cabecera el **token JWT** obtenido durante el login. **Spring Security** intercepta la petición, valida el token y extrae el **id** y el **rol** del usuario antes de que llegue al controlador. A partir de ahí, la petición fluye por las **capas** del backend siguiendo el patrón **Controller → Service → Repository**, hasta llegar a la base de datos MySQL. La respuesta recorre el camino inverso, se serializa en **JSON** y Angular la procesa mediante **RxJS** para actualizar la vista en **tiempo real**.



5.4.- Descomposición Modular E Interrelación Entre Módulos

Orbit Control está estructurado en **tres** capas bien diferenciadas: el **frontend**, el **backend** y la **base de datos**.

Cada capa tiene una responsabilidad clara y se comunica con la siguiente a través de interfaces bien definidas, lo que facilita el mantenimiento y la escalabilidad del proyecto a largo plazo.

En el **frontend**, los módulos están organizados dentro de la carpeta **features/**, donde cada módulo agrupa las vistas, componentes y servicios relacionados. Todos los módulos comparten componentes comunes a través de **shared/**, como tablas, modales, formularios o el sidebar de navegación, y acceden a servicios globales y guardas de ruta mediante **core/**.

Cada petición que sale del frontend pasa por el interceptor **HTTP**, que añade automáticamente el **token JWT** en la cabecera de autorización.

En el **backend**, las peticiones llegan a los **controladores**, que se encargan de validar la entrada y delegar la lógica de negocio en los **servicios**. Los servicios acceden a la **base de datos** a través de los **repositorios JPA**.

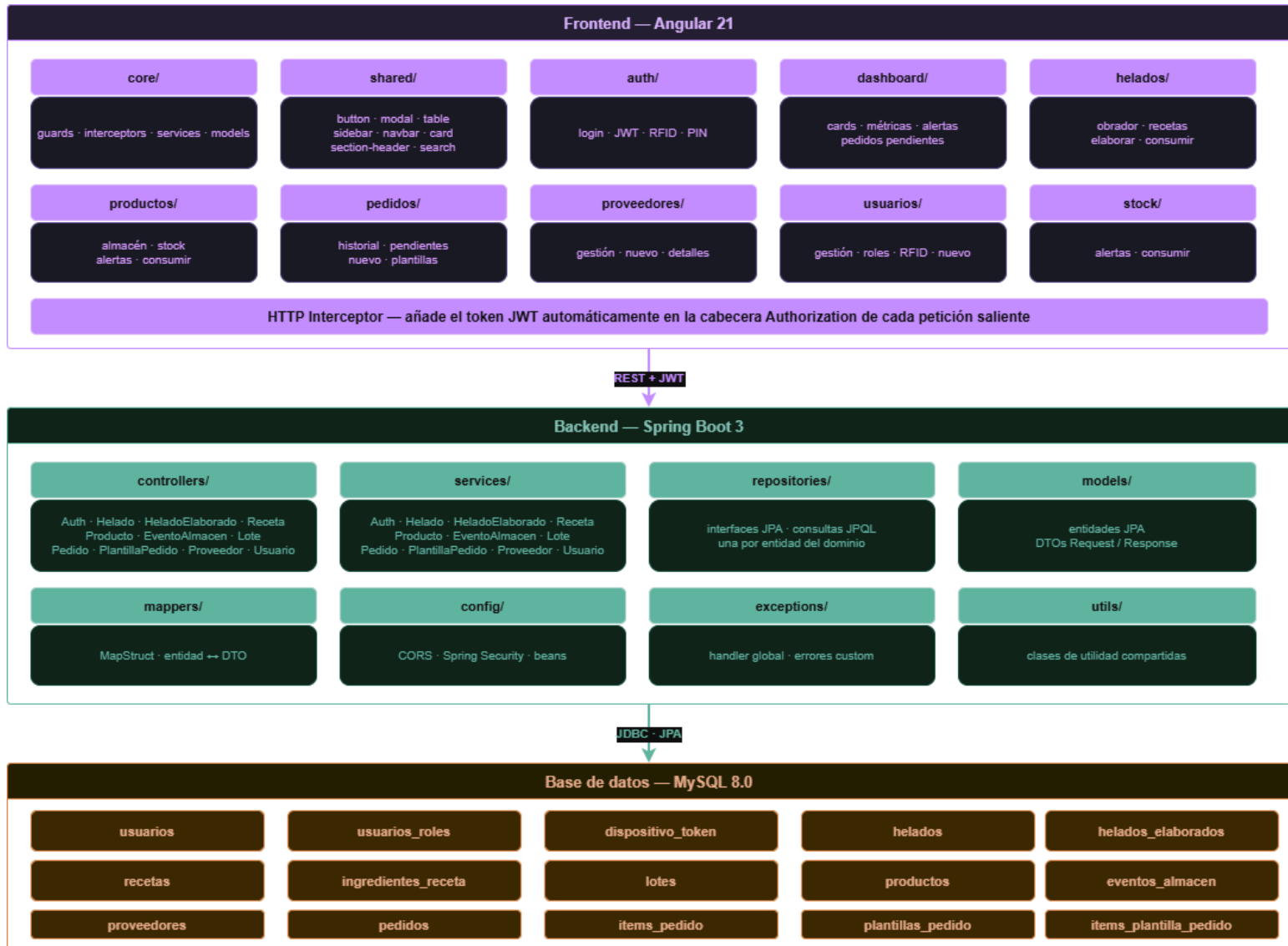
Los datos se transforman entre **entidades** y **DTOs** mediante **MapStruct**, evitando exponer directamente el modelo de base de datos.

La base de datos está compuesta por **15** tablas que cubren todos los procesos de la aplicación: la gestión de usuarios y autenticación, el obrador con recetas, helados y elaboraciones, el almacén con productos y sus movimientos de stock, y el ciclo de pedidos con proveedores.

La seguridad, la configuración **CORS** y el manejo centralizado de errores están en sus propios paquetes, separados del resto de la lógica de negocio.



El siguiente diagrama muestra la estructura completa de módulos y la interrelación entre las tres capas.



5.4.1.- Estructura de directorios

El repositorio está dividido en dos carpetas raíz independientes, **backend/** y **frontend/**, cada una con su propia estructura interna organizada según las convenciones de su tecnología.

5.4.1.1.- Backend

El backend sigue la estructura estándar de un proyecto Maven con Spring Boot.

Todos los paquetes de la aplicación están dentro de **src/main/java/rra/orbitcontrol/** y cada uno tiene una responsabilidad única y acotada:

- **config/** centraliza la configuración de seguridad y CORS
- **controllers/** expone los endpoints REST
- **services/** contiene toda la lógica de negocio
- **repositories/** gestiona el acceso a datos mediante Spring Data JPA.
- **models/** se organizan en tres subpaquetes diferenciados:
 - **entities/** para las clases JPA
 - **dtos/** con separación explícita entre payloads de entrada y salida
 - **enums/** para los tipos enumerados compartidos.
- **mappers/** centraliza las transformaciones entre entidades y DTOs mediante MapStruct
- **exceptions/** define el handler global de errores
- **utils/** agrupa servicios transversales como **JwtService**.

```
backend/
├── src/main/java/rra/orbitcontrol/
│   ├── config/           # SecurityConfig, JwtConfig, CORS
│   ├── controllers/      # REST endpoints (@RestController)
│   ├── services/         # Lógica de negocio
│   ├── repositories/     # Spring Data JPA
│   ├── models/
│   │   ├── entities/     # @Entity -- tablas JPA
│   │   ├── dtos/
│   │   │   ├── requests/ # Payload entrada (LoginRequest,
etc.)
│   │   │   │             # Payload salida (AuthResponse, etc.)
│   │   │   └── responses/
│   │   └── enums/        # TipoProducto, TipoEntidad, etc.
│   ├── mappers/          # MapStruct entity ↔ DTO
│   ├── exceptions/       # GlobalExceptionHandler
│   └── utils/             # JwtService, helpers
```

5.4.1.2.- Frontend

El frontend sigue la estructura recomendada para proyectos Angular con componentes standalone.

Dentro de **src/app/** conviven tres carpetas principales con responsabilidades bien diferenciadas.

- **core/** agrupa todo lo que es transversal a la aplicación: las interfaces TypeScript que modelan el dominio, los servicios HTTP, las guardas de ruta y el interceptor que adjunta el token JWT en cada petición saliente.
- **features/** organiza cada módulo de negocio de forma autónoma, de manera que cada carpeta contiene únicamente las vistas y componentes relacionados con esa área.
- **shared/** centraliza todos los componentes reutilizables de la interfaz, como tablas, modales, notificaciones, el sidebar y el header de sección, que se consumen desde cualquier módulo sin duplicar código.



```
frontend/
├── src/app/
│   ├── core/
│   │   ├── models/           # Interfaces TS (Usuario, Pedido, etc.)
│   │   ├── services/        # HTTP services (auth, pedido, helado...)
│   │   ├── guards/          # roleGuard, authGuard
│   │   └── interceptors/    # authInterceptor (añade JWT)
│   ├── features/
│   │   ├── auth/login/      # Pantalla login (3 métodos)
│   │   ├── dashboard/      # Dashboard + cards
│   │   ├── helados/
│   │   │   ├── stock/
│   │   │   ├── elaboracion/
│   │   │   ├── recetas/
│   │   │   └── dashboard-card/
│   │   ├── pedidos/
│   │   │   ├── dashboard-card/
│   │   │   └── pedidos-pendientes/
│   │   ├── productos/
│   │   ├── proveedores/
│   │   ├── stock/dashboard-card/
│   │   └── usuarios/
│   └── shared/
│       ├── table/
│       ├── card/
│       ├── modal/
│       ├── confirm/
│       ├── notification/
│       ├── search/
│       ├── sidebar/ navbar/
│       └── section-header/
```

5.5.- Descripción De Los Módulos Del Proyecto

5.5.1.- Módulos Frontend

El frontend de Orbit Control está organizado en módulos independientes.

Todos comparten los componentes comunes de **shared/** y los servicios globales de **core/**, lo que evita duplicar código y mantiene una interfaz visual consistente en toda la aplicación.

5.5.1.1 Módulo de Autenticación (auth/)

El módulo de **autenticación** es el punto de entrada a la aplicación. Se encarga de gestionar el acceso de los usuarios mediante tres métodos distintos:

- Contraseña alfanumérica
- PIN numérico
- Tarjeta NFC

Esta variedad de métodos responde a una necesidad real del sector, ya que los empleados del obrador trabajan habitualmente con las **manos ocupadas** y necesitan una forma de acceder a la aplicación de forma rápida y sin necesidad de un teclado.

Una vez el usuario se autentica correctamente, el backend devuelve un **token JWT** que el módulo almacena y gestiona durante toda la sesión.

A partir de ese momento, el interceptor HTTP incluido en **core/** adjunta el token automáticamente en la cabecera de cada petición que sale del frontend, sin necesidad de hacerlo manualmente en cada llamada:

```
// auth.interceptor.ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getToken();
  if (!token) return next(req);
  return next(req.clone({
    headers: { Authorization: `Bearer ${token}` }
  }));
};
```

Sistema de permisos

El control de acceso de Orbit Control funciona con dos capas independientes que actúan a la vez.

La primera capa es el **rol**, que determina a qué secciones puede navegar el usuario.

- **HELADERO** solo tiene acceso al obrador y las recetas.
- **ENCARGADO** accede a pedidos, proveedores y almacén.
- **DEPENDIENTE** solo puede consultar stock, historial de pedidos y alertas, sin acceso al obrador ni a la gestión.
- **ADMIN** tiene acceso a todo, incluida la gestión de usuarios.

Esta capa se implementa en el frontend mediante guardas de ruta. Cada ruta tiene declarados los roles que pueden activarla, y si el usuario no tiene ninguno de ellos es redirigido automáticamente al dashboard:

```
// role.guard.ts
export const roleGuard = (roles: string[]): CanActivateFn => () =>
{
  const auth = inject(AuthService);
  const router = inject(Router);
  const hasRole = roles.some(r => auth.hasRole(r));
  return hasRole ? true : router.createUrlTree(['/dashboard']);
};

// app.routes.ts
{ path: 'obrador/elaborar', component: ElaborarHeladoComponent,
  canActivate: [roleGuard(['ROLE_ADMIN', 'ROLE_HELADERO'])] },
{ path: 'pedidos/nuevo', component: NuevoPedidoComponent,
  canActivate: [roleGuard(['ROLE_ADMIN', 'ROLE_ENCARGADO'])] },
{ path: 'usuarios', component: GestionarUsuariosComponent,
  canActivate: [roleGuard(['ROLE_ADMIN'])] },
```

La segunda capa es el **método de login**, que determina si el usuario puede escribir dentro de las secciones a las que tiene acceso.

Iniciar sesión con **tarjeta NFC** otorga **acceso completo** de escritura. Iniciar sesión con PIN deja al usuario en **modo lectura**, aunque su rol le permita acceder a esa sección. La excepción es el **ADMIN**, que siempre tiene acceso de escritura independientemente de cómo inicie sesión.

En el frontend esto se gestiona con el método **hasPermission()**, que comprueba el método de login extraído del propio token JWT:

```
// auth.service.ts
getMetodoAcceso(): string | null {
  const token = this.getToken();
  if (!token) return null;
  return this.jwtHelper.decodeToken(token)?.metodoAcceso ?? null;
}

isNfcLogin(): boolean { return this.getMetodoAcceso() === 'NFC'; }
hasPermission(): boolean { return this.isNfcLogin() ||
this.hasRole('ROLE_ADMIN'); }
```

En el backend, el **filtro JWT** extrae el método de acceso del token y lo inyecta como una authority adicional (**METHOD_PIN** o **METHOD_NFC**), de forma que Spring Security puede aplicar ambas capas en cada petición de forma centralizada.

```
// JwtAuthFilter.java -- extrae el método del token e inyecta la authority
String metodo = claims.get("metodoAcceso", String.class); // "PIN" o "NFC"
authorities.add(new SimpleGrantedAuthority("METHOD_" + metodo));

// SecurityConfig.java -- aplica ambas capas en cada request
.requestMatchers(HttpMethod.GET, "/api/**").hasAnyAuthority("METHOD_PIN",
"METHOD_NFC")
.anyRequest().hasAnyAuthority("METHOD_NFC", "ROLE_ADMIN")
```


El resultado práctico es que un **ENCARGADO** que ficha con NFC puede crear pedidos y gestionar proveedores con total libertad, mientras que ese mismo **ENCARGADO** que entra con PIN puede ver todo pero no modificar nada.

El **DEPENDIENTE** que inicia sesión con NFC puede recibir pedidos y registrar consumos, pero no tiene acceso al obrador ni a la gestión de almacén independientemente del método de login.

Si el **DEPENDIENTE** inicia sesión con PIN, solo puede consultar stock, historial y alertas.

5.5.1.2.- Módulo de Dashboard (dashboard/)

El dashboard es la pantalla principal que ve el usuario al iniciar sesión. Su función es ofrecer una **visión rápida** del estado del negocio sin necesidad de entrar en cada módulo por separado.

La información que muestra **varía** según el **rol del usuario**, pero en general incluye tres bloques principales: los **pedidos pendientes** de recibir ordenados por fecha de entrega, los **últimos movimientos del obrador** y el **almacén** con las elaboraciones y consumos más recientes, y las **alertas de stock** tanto de helados como de productos por debajo del mínimo configurado.

Desde el dashboard el usuario también puede acceder directamente a las acciones **más habituales** del día a día, como crear un nuevo pedido o consultar el stock del obrador, sin tener que navegar por el menú lateral.

5.5.1.3.- Módulo de Helados (helados/)

El módulo de helados es el **núcleo operativo** de la aplicación y el que más uso tiene en el día a día del obrador. Agrupa todo lo relacionado con la producción de helados: el catálogo de helados, las recetas, las elaboraciones y el control de stock.

Desde la vista de stock el usuario puede consultar el listado completo de helados con su cantidad actual y mínima, con indicadores visuales que destacan aquellos que están por debajo del **mínimo configurado**. La vista de alertas muestra únicamente los helados en estado crítico, permitiendo enviar notificaciones push al equipo directamente desde la aplicación.

El flujo de elaboración es el más importante del módulo. El usuario selecciona el helado que quiere producir, introduce la cantidad y la fecha de elaboración, y el sistema crea automáticamente los registros correspondientes en la tabla **helados_elaborados** con estado **ALMACEN**. Paralelamente, la vista de consumo permite registrar las salidas de helado del obrador, descontando el stock actual.

Por último, el módulo incluye la gestión completa de recetas, donde se definen los ingredientes y cantidades necesarios para elaborar cada sabor. Las recetas soportan subrecetas, es decir, una receta puede incluir otra receta como ingrediente, lo que permite reutilizar bases y mezclas intermedias sin duplicar información.

5.5.1.4.- Módulo de Productos (productos/)

El módulo de productos gestiona el almacén de materias primas de la heladería. Es el encargado de **mantener el control del stock** de todos los ingredientes y productos necesarios para la producción del obrador y la tienda.

La vista principal muestra un listado paginado de todos los productos con su stock actual, stock mínimo, precio, proveedor asociado y unidad de medida. Los productos pueden filtrarse por tipo: obrador, tienda o ambos, lo que permite separar fácilmente los ingredientes de producción de los productos de venta directa.

El módulo incluye una vista de alertas que muestra únicamente los productos por debajo del mínimo configurado, desde la que también se pueden enviar notificaciones push al equipo responsable de realizar los pedidos.

La vista de consumo permite registrar salidas manuales de stock, por ejemplo por mermas o ajustes de inventario, descontando la cantidad del stock actual. Si la cantidad a consumir supera el stock disponible, el sistema deshabilita el botón de confirmación y muestra un mensaje de error.

Cada producto tiene además una vista de detalle donde se pueden consultar y añadir los lotes asociados, lo que permite tener trazabilidad completa sobre el origen de cada unidad de stock registrada.

5.5.1.5.- Módulo de Pedidos (pedidos/)

El módulo de pedidos gestiona el ciclo completo de compras a proveedores, desde la creación del pedido hasta su recepción y pago. Es accesible para los roles **ADMIN** y **ENCARGADO**, mientras que **HELADERO** y **DEPENDIENTE** pueden recibir pedidos, consultar el historial y los pedidos pendientes pero no crear ni modificar pedidos.

La creación de un nuevo pedido se realiza mediante un formulario donde el usuario selecciona el proveedor, añade los ítems con su producto y cantidad, y establece una fecha de entrega obligatoria que no puede ser anterior al día actual.

Si el usuario abandona el formulario sin confirmar el pedido, el sistema **guarda automáticamente un borrador** para que no se pierda el trabajo realizado.

El módulo incluye dos vistas de seguimiento diferenciadas.

- La **vista de historial** muestra todos los pedidos con su estado representado mediante colores, permitiendo filtrar y consultar el detalle de cada uno.
- La **vista de pendientes** se divide en dos pestañas: entregas, con los pedidos en estado enviado ordenados por fecha de entrega próxima, y pagos, con los pedidos recibidos pendientes de liquidar.

Al marcar un pedido como recibido, el sistema abre un modal para registrar los lotes recibidos por ítem, actualizando automáticamente el stock del almacén.

Por último, el módulo incluye la gestión de plantillas de pedido, que permiten preconfigurar combinaciones habituales de productos por proveedor para agilizar la creación de pedidos recurrentes.

5.5.1.6.- Módulo de Proveedores (proveedores/)

El módulo de proveedores gestiona el catálogo de empresas suministradoras de la heladería. Al igual que el módulo de pedidos, es accesible únicamente para los roles **ADMIN** y **ENCARGADO**.

La vista principal muestra un listado paginado de todos los proveedores con su nombre, NIF, email, teléfono, tipo de producto que suministran y estado activo. Desde aquí se puede filtrar por tipo de producto y navegar al detalle de cada proveedor.

El formulario de creación y edición recoge los datos de contacto del proveedor: nombre, NIF, email, teléfono, dirección, tipo de producto que suministra.

El tipo de producto puede ser obrador, tienda o ambos, lo que permite asociar cada proveedor con el área del negocio a la que abastece.

5.5.1.7.- Módulo de Usuarios (usuarios/)

El módulo de usuarios es accesible únicamente para el rol **ADMIN**. Desde aquí se gestiona todo lo relacionado con el acceso y los permisos del equipo de la heladería.

La vista principal muestra un listado paginado de todos los usuarios con su nombre, nombre de usuario, correo y roles asignados representados mediante badges de color. Desde aquí el administrador puede filtrar por rol, crear nuevos usuarios y acceder al detalle de cada uno para editarlo.

El formulario de creación recoge los datos personales del usuario: nombre de usuario, nombre y apellidos, email, teléfono y DNI. El email es un campo único y obligatorio, mientras que el DNI es único pero opcional.

La asignación de tarjeta RFID se puede realizar directamente desde el formulario de creación o posteriormente desde el listado principal. En ambos casos el proceso consiste en abrir un modal de espera, acercar la tarjeta al lector y el sistema registra automáticamente el UID de la tarjeta y lo asocia al usuario.

5.5.2.- Módulos Backend

El backend de Orbit Control sigue una arquitectura en capas basada en el patrón **Controller** → **Service** → **Repository**, donde cada capa tiene una responsabilidad única y bien definida.

Esta separación facilita el mantenimiento del código, permite testear cada capa de forma independiente y hace que el proyecto sea fácilmente escalable.

5.5.2.1.- Controllers

Los controladores son el punto de entrada de la **API REST**.

Se encargan de recibir las peticiones **HTTP**, validar los datos de entrada mediante **DTOs** y delegar la lógica de negocio en los servicios correspondientes.

Cada controlador está asociado a un dominio concreto del negocio: autenticación, helados, elaboraciones, recetas, productos, eventos de almacén, lotes, pedidos, plantillas de pedido, proveedores y usuarios.

Los controladores nunca contienen lógica de negocio, su única responsabilidad es actuar como intermediarios entre el cliente y los servicios.

5.5.2.2.- Services

Los servicios contienen toda la lógica de negocio de la aplicación.

Es en esta capa donde se aplican las reglas del dominio, como el descuento automático de stock al registrar una elaboración, la validación de fechas en los pedidos o la gestión del ciclo de vida de los lotes.

Cada servicio corresponde a un dominio del negocio y se comunica con uno o varios repositorios para acceder a los datos.

Los servicios también utilizan los **mappers** de **MapStruct** para transformar las entidades **JPA** en **DTOs** de respuesta antes de devolver los datos al controlador.

5.5.2.3.- Repositories

Los repositorios son la capa de **acceso a datos**.

Extienden las interfaces de **Spring Data JPA**, lo que permite realizar operaciones **CRUD** básicas sin escribir código adicional.

Para las consultas más complejas se definen métodos personalizados mediante anotaciones **JPQL**, manteniendo toda la lógica de base de datos encapsulada en esta capa y sin filtrar hacia las capas superiores.

5.5.2.4.- Mappers

Los mappers se implementan con **MapStruct** y se encargan de transformar automáticamente las entidades **JPA** en **DTOs** de respuesta y viceversa.

Esto evita exponer directamente el modelo de base de datos al cliente y permite controlar exactamente qué información se devuelve en cada endpoint.

5.5.2.5.- Config Y Excepciones

El paquete **config/** centraliza la configuración de **Spring Security**, las reglas **CORS** y los beans de la aplicación.

El paquete **exceptions/** define un manejador global (**GlobalExceptionHandler**) que intercepta todas las excepciones lanzadas por los servicios y las transforma en **respuestas HTTP** con códigos de error y mensajes descriptivos, evitando que los errores internos se expongan directamente al cliente.

Durante el desarrollo se realizó una refactorización completa de este sistema. En su estado inicial, la mayoría de los servicios lanzaban **RuntimeException** genéricas, lo que obligaba al manejador a detectar el tipo de error inspeccionando el texto del mensaje, una solución frágil y difícil de mantener a medida que el proyecto crecía.

La refactorización introdujo tres excepciones tipadas con semántica HTTP clara y bien definida.

- **InsufficientStockException** se lanza cuando no hay stock suficiente para completar una elaboración o consumo, y el manejador la traduce a un **HTTP 422 Unprocessable Entity**.
- **DuplicateEntityException** se lanza al intentar asignar una tarjeta NFC ya registrada en otro usuario, devolviendo un **HTTP 409 Conflict**.
- **EntityNotFoundException** devuelve siempre un **HTTP 404** con un mensaje descriptivo que incluye el tipo de entidad y el identificador buscado.

Además, se añadió una comprobación de existencia previa en todos los métodos de borrado, garantizando que un intento de eliminar un recurso inexistente devuelve un **404** en lugar de completarse silenciosamente.

5.5.3.- Base De Datos

La base de datos de Orbit Control está implementada en *MySQL* y está compuesta por **15 tablas** organizadas en torno a **cuatro áreas funcionales** del negocio: usuarios y autenticación, obrador y producción, almacén y productos, y pedidos y proveedores.

5.5.3.1.- Usuarios Y Autenticación

Este bloque gestiona todo lo relacionado con el acceso a la aplicación.

La tabla **usuarios** almacena los datos personales y las credenciales de cada empleado, incluyendo el *UID* de la tarjeta **NFC** y el **PIN numérico hasheado**.

La tabla **usuarios_rols** gestiona la asignación de roles, permitiendo que un usuario tenga más de uno simultáneamente.

La tabla **dispositivo_token** almacena los **tokens** de Firebase para el envío de notificaciones push a cada dispositivo registrado.

5.5.3.2.- Obrador y producción

Este bloque cubre el ciclo completo de **elaboración de helados**.

La tabla **recetas** define los sabores con su coste de elaboración y tipo.

La tabla **ingredientes_receta** relaciona cada receta con sus ingredientes o subrecetas, permitiendo reutilizar bases dentro de otras recetas.

La tabla **helados** representa el catálogo con su stock actual y mínimo.

La tabla **helados_elaborados** registra cada unidad producida con su fecha de elaboración, fecha de caducidad, estado y código de producción único.

La tabla **lotes** gestiona los lotes de ingredientes y helados mediante una relación polimórfica.

5.5.3.3.- Almacén y productos

Este bloque gestiona las **materias primas**.

La tabla **productos** almacena cada ingrediente con su precio, stock actual, stock mínimo y unidad de medida.

La tabla **eventos_almacen** registra todos los movimientos de stock de cada producto, tanto entradas como salidas, permitiendo tener un historial completo de cada ingrediente.

5.5.3.4.- Pedidos y proveedores

Este bloque cubre el **ciclo de compras**.

La tabla **proveedores** almacena los datos de cada empresa suministradora.

La tabla **pedidos** gestiona cada orden de compra con su estado y fechas.

La tabla **items_pedido** detalla los productos incluidos en cada pedido con su cantidad y precio.

Las tablas **plantillas_pedido** e **items_plantilla_pedido** permiten guardar combinaciones habituales de productos por proveedor para agilizar la creación de pedidos recurrentes.

5.6.- Descripción De La Interfaz De Usuario

La interfaz de Orbit Control sigue una identidad visual propia y consistente en toda la aplicación.

El color principal es el violeta **#7C3AED**, que aparece en la barra de navegación superior, los encabezados de sección, los botones de acción principal y los badges de estado.

La tipografía utilizada es Inter para el cuerpo de texto y Comfortaa para los títulos y la marca.

El fondo general es blanco, lo que contrasta limpiamente con los elementos morados y facilita la lectura en entornos con mucha luz como el obrador.

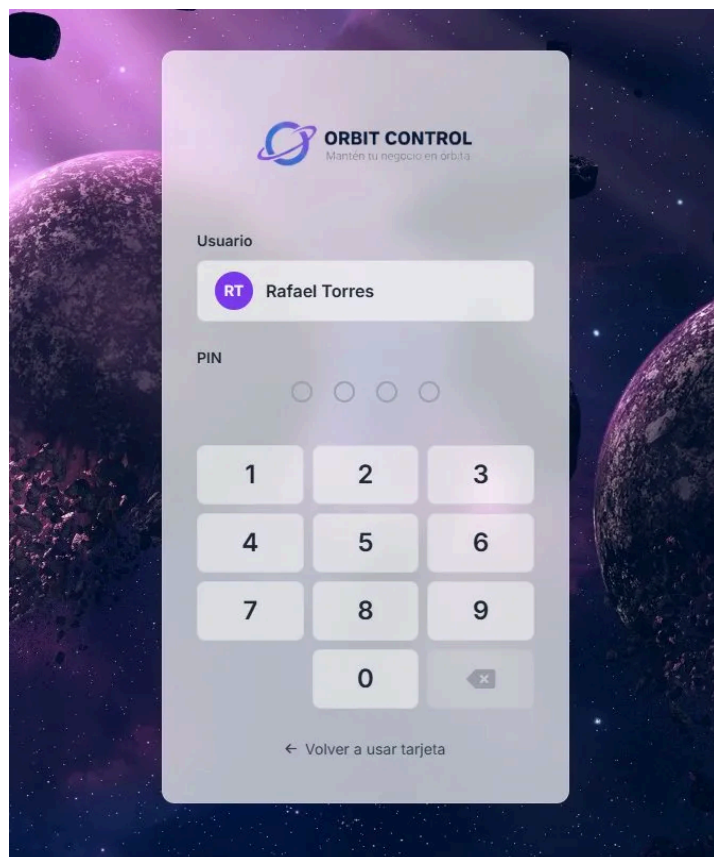
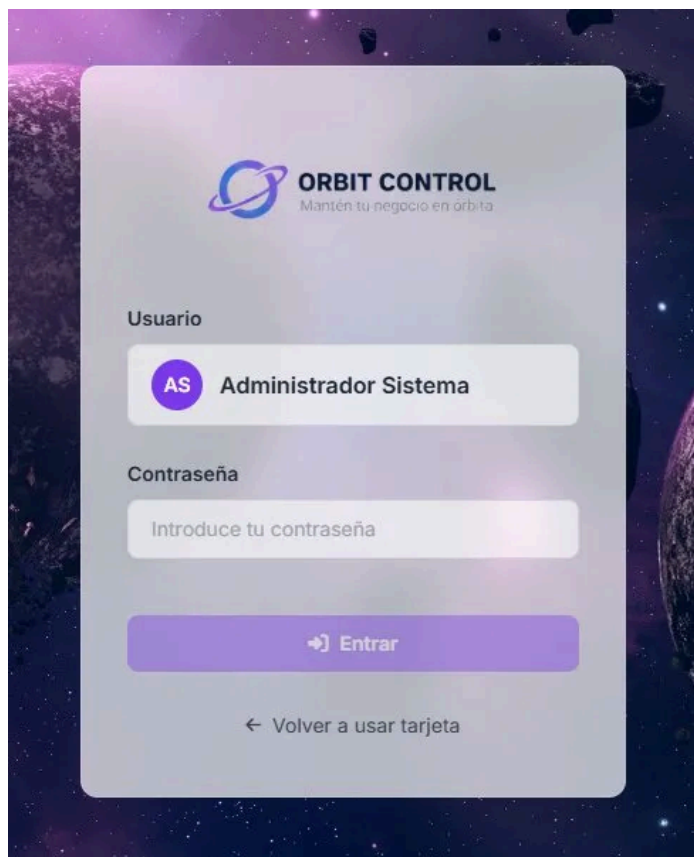
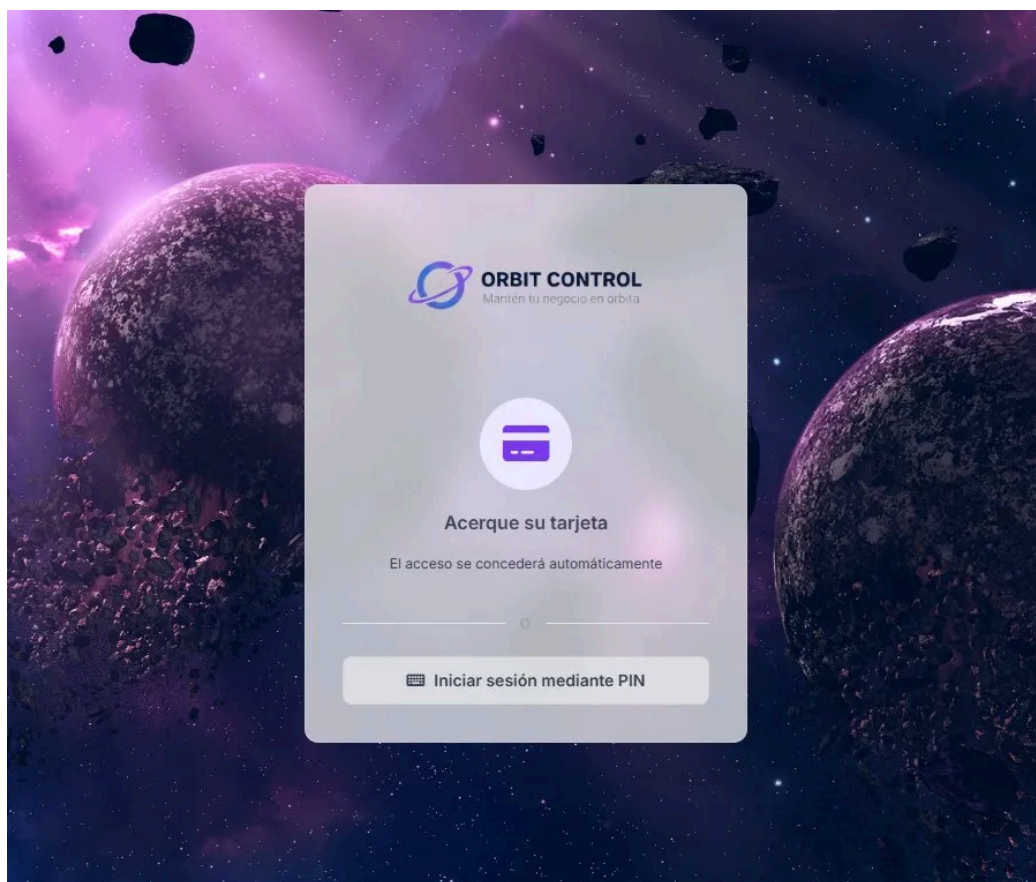
5.6.1.- Pantallas de autenticación

La aplicación arranca mostrando la pantalla de acceso por **tarjeta NFC**, que es el método principal para los empleados del obrador. Muestra el logo de Orbit Control, un icono de tarjeta y el mensaje **"Acerque su tarjeta"**, con una opción para cambiar al acceso por PIN.

Desde la pantalla de **PIN** el usuario selecciona su nombre de usuario e introduce el código mediante un **teclado numérico táctil** en pantalla.

Si el usuario es **"admin"** se activa el acceso por **contraseña alfanumérica**, el formulario muestra el usuario seleccionado y un campo de texto para introducir la contraseña.

Los tres métodos comparten el mismo fondo con imagen espacial, reforzando la identidad visual del producto.

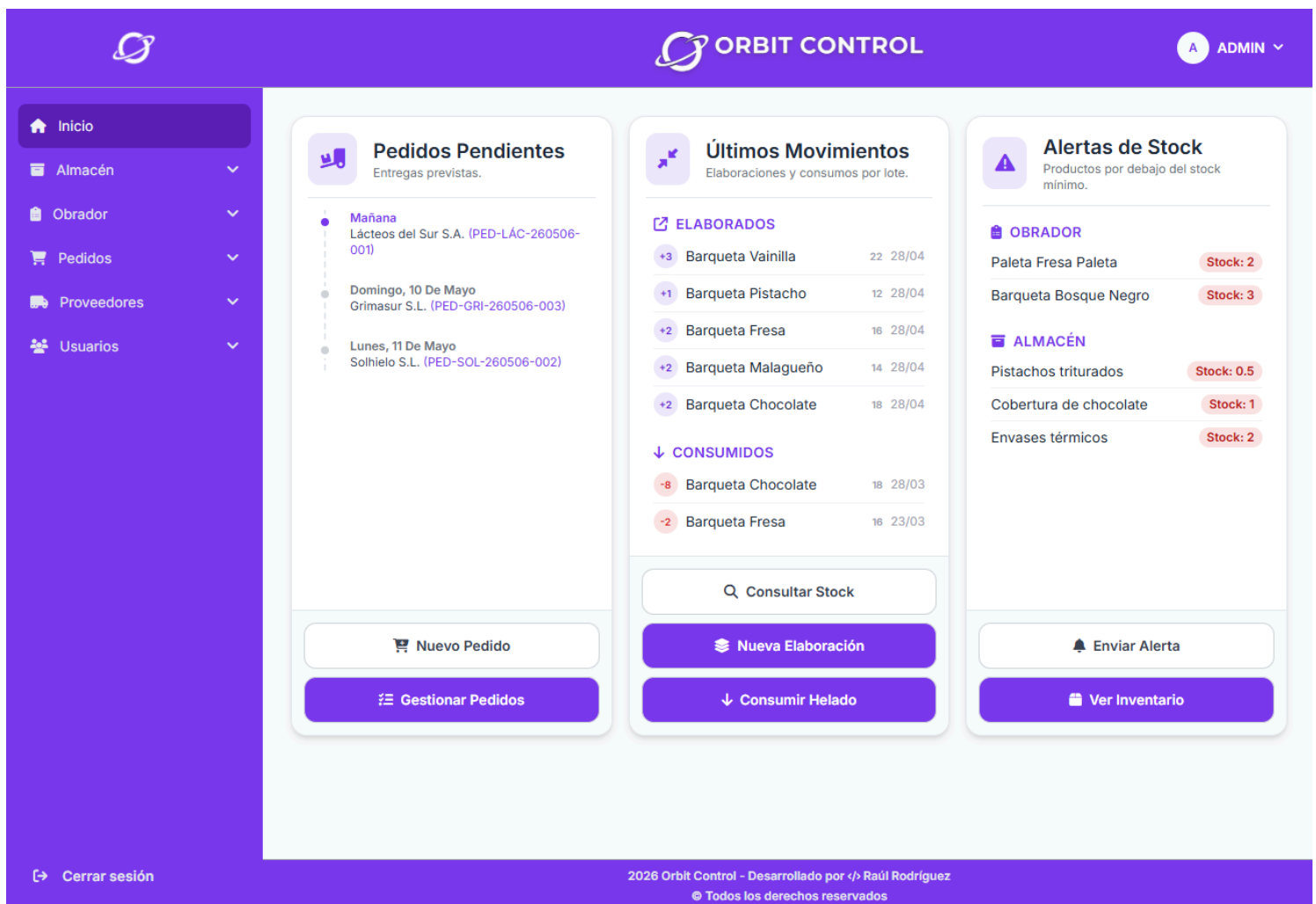


5.6.2.- Dashboard

El dashboard se divide en **tres cards principales** dispuestas en columna: pedidos pendientes con las próximas entregas ordenadas por fecha, últimos movimientos del obrador con las elaboraciones y consumos más recientes, y alertas de stock con los helados e ingredientes por debajo del mínimo.

Cada card incluye accesos directos a las **acciones más habituales**, como crear un nuevo pedido, registrar una elaboración o consumir helado.

La navegación principal se encuentra en el **sidebar** izquierdo, con los módulos agrupados por área.



The dashboard is titled "ORBIT CONTROL" and features a sidebar on the left with navigation links: Inicio, Almacén, Obrador, Pedidos, Proveedores, and Usuarios. The main content area is divided into three columns:

- Pedidos Pendientes** (Entregas previstas):
 - Mañana: Lácteos del Sur S.A. (PED-LÁC-260506-001)
 - Domingo, 10 De Mayo: Grimasur S.L. (PED-GRI-260506-003)
 - Lunes, 11 De Mayo: Solhelo S.L. (PED-SOL-260506-002)
- Últimos Movimientos** (Elaboraciones y consumos por lote):
 - ELABORADOS**:
 - +3 Barqueta Vainilla (22/28/04)
 - +1 Barqueta Pistacho (12/28/04)
 - +2 Barqueta Fresa (16/28/04)
 - +2 Barqueta Malagueño (14/28/04)
 - +2 Barqueta Chocolate (18/28/04)
 - CONSUMIDOS**:
 - 8 Barqueta Chocolate (18/28/03)
 - 2 Barqueta Fresa (16/23/03)
- Alertas de Stock** (Productos por debajo del stock mínimo):
 - OBRADOR**:
 - Paleta Fresa Paleta (Stock: 2)
 - Barqueta Bosque Negro (Stock: 3)
 - ALMACÉN**:
 - Pistachos triturados (Stock: 0.5)
 - Cobertura de chocolate (Stock: 1)
 - Envases térmicos (Stock: 2)

At the bottom of each card are action buttons: "Nuevo Pedido" and "Gestionar Pedidos" for the first card; "Consultar Stock", "Nueva Elaboración", and "Consumir Helado" for the second; and "Enviar Alerta" and "Ver Inventario" for the third.

Footer: 2026 Orbit Control - Desarrollado por </> Raúl Rodríguez. © Todos los derechos reservados.



5.6.3.- Vistas de Listados

Las vistas de listado, como la gestión del almacén, presentan los datos en una tabla paginada con **columnas ordenables** y una **barra de búsqueda** en la parte superior.

Cada fila incluye acciones de edición y eliminación.

Al acceder desde un dispositivo móvil, las tablas se transforman automáticamente en **cards**, adaptando la presentación de la información al tamaño de pantalla sin perder ningún dato.

Gestionar Almacén
Administra los productos del almacén

[+ Nuevo producto](#)

Tipo de producto

Ordenar por...

Almendras molidas
Comprital España S.L.

PRECIO(€)/UD12.5€

STOCK ACTUAL2,00kg (2 BOLSA)

STOCK MÍNIMO1,00kg

CANTIDAD POR ENVASE1kg

TIPO DE PRODUCTO

OBRADOR

[✎ Editar](#)

[🗑 Eliminar](#)

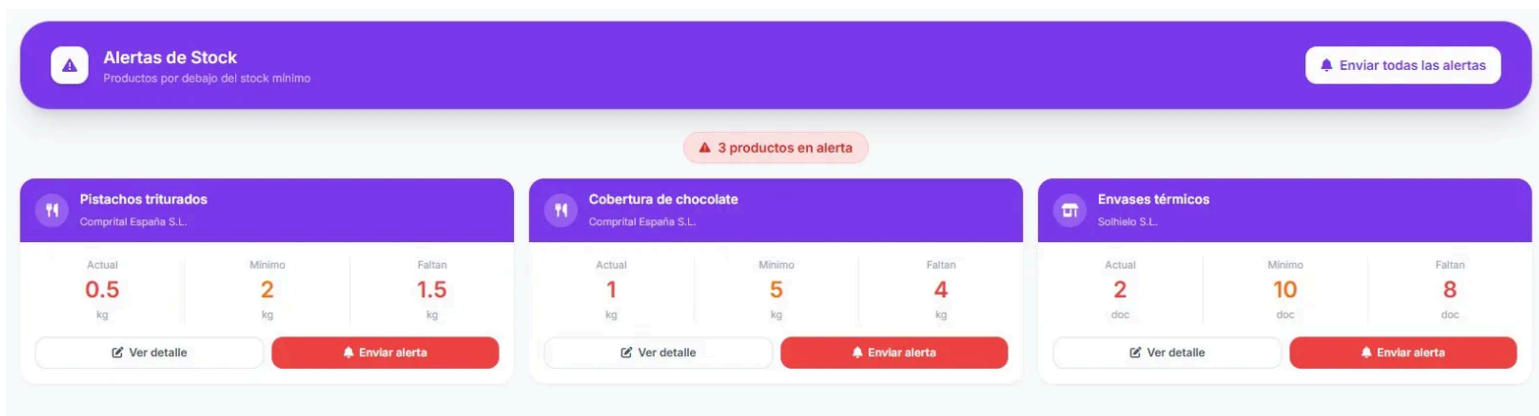
Gestionar Almacén Administra los productos del almacén + Nuevo producto							
Buscar... Tipo de producto Ordenar por...							
TIPO DE PRODUCTO	NOMBRE	PROVEEDOR	PRECIO(€)/UD	STOCK ACTUAL	STOCK MÍNIMO	CANTIDAD POR ENVASE	ACCIONES
OBRADOR	Almendras molidas	Comprital España S.L.	12,5€	2,00kg (2 BOLSA)	1,00kg	1kg	✎ 🗑
OBRADOR	Avellanas troceadas	Comprital España S.L.	14€	3,00kg (3 BOLSA)	1,00kg	1kg	✎ 🗑
OBRADOR	Azúcar	Grimasur S.L.	0,95€	30,00kg (~1 SACO)	8,00kg	25kg	✎ 🗑
OBRADOR	Base Grimasur	Grimasur S.L.	5,8€	12,00kg (~2 BOLSA)	4,00kg	5kg	✎ 🗑
TIENDA	Bolsas kraft	Solhielo S.L.	0,12€	300,00ud (3 CAJA)	80,00ud	100ud	✎ 🗑
OBRADOR	Cacao en polvo 70%	Comprital España S.L.	12,5€	4,00kg (4 BOTE)	1,00kg	1kg	✎ 🗑

5.6.4.- Vistas de Alertas

Las vistas de alertas presentan la información en formato de **cards** en lugar de tabla, ya que el objetivo es dar **visibilidad inmediata** al problema.

Cada card muestra el nombre del producto, el proveedor, y tres datos numéricos destacados: stock actual, mínimo y unidades que faltan.

Desde cada card se puede ir al detalle del producto o enviar una **alerta push** individual, y desde el encabezado de la página se pueden enviar **todas** las alertas de golpe.



The screenshot displays a dashboard titled 'Alertas de Stock' (Stock Alerts) with a subtitle 'Productos por debajo del stock mínimo' (Products below minimum stock). A button 'Enviar todas las alertas' (Send all alerts) is in the top right. A status bar indicates '3 productos en alerta' (3 products in alert). Three product cards are shown, each with a header, product name, supplier, and three data points: Actual, Minimum, and Missing. Each card has 'Ver detalle' (View detail) and 'Enviar alerta' (Send alert) buttons.

Producto	Proveedor	Actual	Mínimo	Faltan
Pistachos triturados	Comprital España S.L.	0.5 kg	2 kg	1.5 kg
Cobertura de chocolate	Comprital España S.L.	1 kg	5 kg	4 kg
Envases térmicos	Solihalo S.L.	2 doc	10 doc	8 doc

5.6.5.- Vista de Pedidos

La vista de pedidos pendientes organiza los pedidos en **dos secciones** diferenciadas: pendientes de entrega en azul y pendientes de pago en verde.

Cada pedido se muestra como una **card** con el proveedor, el código de pedido, las fechas y el importe total.

Los pedidos con fecha de entrega vencida muestran la fecha en **rojo**, alertando visualmente del retraso.

Pedidos Pendientes

Entregas y pagos pendientes

+ Nuevo pedido

PENDIENTES DE ENTREGA (3)

Lácteos del Sur S.A.

PED-LAC-260506-001

Pendiente

Pedido

06 may

Entrega esperada

08 may

en 2 días

Reposición urgente de leche entera y nata 35%.

3 artículos

68,20 €

Solhelo S.L.

PED-SOL-260506-002

Pendiente

Pedido

06 may

Entrega esperada

11 may

en 5 días

Reposición de envases y material de presentación.

4 artículos

55,00 €

Grimasur S.L.

PED-GR1-260506-003

Pendiente

Pedido

06 may

Entrega esperada

10 may

en 4 días

Bases y endulzantes para temporada de verano.

3 artículos

146,75 €

PENDIENTES DE PAGO (2)

Fabri Ibérica S.L.

PED-FAB-260506-004

Recibido

Pedido

06 may

Entrega esperada

02 may

hace 4d

Variegatos y pulpas de fruta temporalda.

4 artículos

113,60 €

Comprital España S.L.

PED-COM-260506-005

Recibido

Pedido

06 may

Entrega esperada

28 abr

hace 8d

Pastas de sabor y cobertura de chocolate.

4 artículos

229,00 €

5.6.6.- Formularios

Los formularios de creación y edición siguen un diseño limpio y consistente, con los campos agrupados por secciones y etiquetados claramente.

Los campos obligatorios están marcados con un **asterisco** y los opcionales lo indican explícitamente.

El botón de acción principal aparece **deshabilitado** hasta que todos los campos obligatorios estén correctamente rellenados, evitando envíos incompletos.

 **Nuevo Proveedor**
Formulario para registrar un nuevo proveedor

DATOS DEL PROVEEDOR

Nombre *

 Ej: Proveedor S.A.

NIF (opcional)

 Ej: 12345678A

Correo Electrónico *

 proveedor@gmail.com

Teléfono Móvil *

 600 00 00 00

Dirección *

 Ej: Calle Principal, 123, Ciudad

Tipo de Productos

Selecciona... 

 Registrar Proveedor

Limpiar Formulario

5.7.- Descripción De Listados e Informes

En la versión actual de Orbit Control no se generan informes exportables en formatos como PDF o Excel. La consulta de datos se realiza a través de los listados en pantalla disponibles en cada módulo, descritos en el apartado anterior.

La generación de informes exportables está contemplada como una mejora futura, tal y como se detalla en el apartado 9 de este documento.

5.8.- Manual De Usuario

Este manual describe los pasos básicos para utilizar Orbit Control desde el punto de vista del usuario final, sin necesidad de conocimientos técnicos previos.

5.8.1.- Acceso a la aplicación

Para acceder a Orbit Control abre el navegador y entra en tfg.orbitcontrol.es, o abre la **PWA** si la tienes instalada en tu dispositivo.

La pantalla de inicio muestra por defecto el acceso por tarjeta **NFC**. Si tienes tarjeta asignada, acércala al lector y el acceso se concederá automáticamente.

Si prefieres acceder por PIN, pulsa **"Iniciar sesión mediante PIN"**, selecciona tu usuario e introduce el código numérico.

Los usuarios con rol **ADMIN** acceden mediante contraseña alfanumérica en lugar de PIN, lo que añade una **capa adicional de seguridad** para las cuentas con acceso total al sistema.



5.8.2.- Navegación General

Una vez dentro, el **menú lateral izquierdo** da acceso a todos los módulos de la aplicación: Almacén, Obrador, Pedidos, Proveedores y Usuarios.

Cada módulo se despliega mostrando sus **secciones**.

La barra superior muestra el logo de Orbit Control, el nombre de la aplicación y el **usuario conectado** con su rol.

Desde el dashboard puedes acceder directamente a las acciones **más habituales** sin necesidad de navegar por el menú.

5.8.3.- Cómo registrar una elaboración

1. Accede al módulo **Obrador** desde el menú lateral y pulsa **Elaborar Productos**.
2. Selecciona el **tipo** de helado (Barqueta o Paleta)
3. Selecciona el **helado** que quieres producir del desplegable.
4. Introduce la **cantidad a elaborar**.
5. Pulsa **Confirmar** elaboración. El sistema creará automáticamente los registros de producción y **actualizará el stock**.

5.8.4.- Cómo gestionar el almacén

1. Accede al módulo **Almacén** → **Gestionar Almacén**.
2. Usa la **barra de búsqueda** o los **filtros** para encontrar el producto que necesitas.
3. Para editar un producto pulsa el **icono de edición** de su fila.
4. Para eliminar un producto pulsa el **icono de eliminación** de su fila y acepta el modal de confirmación que aparece.
5. Para crear un nuevo producto pulsa **Nuevo producto** en la esquina superior derecha y rellena el formulario.
6. Los productos con stock por debajo del mínimo aparecen destacados en rojo. Puedes ver todos los productos en alerta desde **Almacén** → **Alertas de Stock**.

5.8.5.- Cómo crear un pedido

1. Accede al módulo **Pedidos** → **Nuevo Pedido**, o pulsa el botón **Nuevo Pedido** desde el dashboard.
2. Selecciona el **proveedor** al que quieres hacer el pedido.
3. Añade los productos que necesitas con su **cantidad**.
4. Introduce la **fecha de entrega prevista**.
5. Pulsa **Confirmar pedido** para enviarlo. Si sales sin confirmar, el sistema **guardará automáticamente un borrador**.
6. Puedes hacer seguimiento del pedido desde **Pedidos** → **Pendientes**, donde aparecerá en la sección de entregas pendientes.

5.8.6.- Cómo gestionar usuarios (ADMIN)

1. Accede al módulo **Usuarios** → **Gestión de usuarios**.
2. Para crear un nuevo usuario pulsa **Nuevo usuario** e introduce sus datos: nombre, email, rol y método de autenticación.
3. Para asignar una **tarjeta NFC** a un usuario, pulsa el botón de **asignación NFC** en el listado, espera a que aparezca el modal y acerca la tarjeta al lector.
4. Para editar un usuario pulsa el **icono de edición** de su fila.
5. Para eliminar un usuario pulsa el **icono de eliminación** de su fila y acepta el modal de confirmación que aparece.



5.9.- Manual De Instalación y Despliegue

Este apartado describe los requisitos, la configuración y los comandos necesarios para instalar el proyecto en local y desplegarlo en el entorno TFG.

Requisitos previos

Para trabajar con el proyecto en local es necesario tener instalado lo siguiente:

- **Java 21** (OpenJDK)
- **Node.js 20+**
- **Maven**
- **Angular CLI 21**

Instalación en local

Descarga o clona el repositorio y accede a la rama TFG:

```
git clone https://github.com/raaulroodriguez/OrbitControlTFG.git  
git checkout tfg
```

Instala las dependencias del frontend:

```
npm install
```

Arranque en local

El proyecto incluye un `package.json` en la raíz que centraliza todos los comandos de desarrollo y despliegue, evitando tener que arrancar el frontend y el backend por separado desde distintas terminales. Todos los comandos se ejecutan desde la raíz del proyecto con `npm run <comando>`.

Para arrancar frontend y backend simultáneamente:

```
npm run dev
```

Internamente ejecuta:

```
concurrently --names "BACK,FRONT" \  
  --prefix-colors "bgBlue.bold,bgMagenta.bold" \  
  \"npm run back:quiet\  
  \"npm run front\"
```

Se usa `concurrently` para lanzar en paralelo el **backend** con Spring Boot y el **frontend** con Angular. Los logs se muestran en la misma terminal diferenciados por color: **azul** para el **backend** y **magenta** para el **frontend**.

Para arrancar solo el backend:

```
npm run back
```

Internamente ejecuta:

```
./mvnw spring-boot:run
```

El backend queda disponible en <http://localhost:8080>.



Para arrancar solo el frontend:

```
npm run front
```

Internamente ejecuta:

```
npm start --prefix frontend
```

El frontend queda disponible en <http://localhost:4200>, cualquier cambio en el código se refleja automáticamente en el navegador sin necesidad de reiniciar.

Infraestructura del servidor TFG

El entorno **TFG** está desplegado en un servidor **Hetzner CX43** con Ubuntu 24.04. La arquitectura del servidor es la siguiente:

- Nginx actúa como reverse proxy, recibiendo las peticiones HTTPS en el puerto **443** bajo el dominio api-tfg.orbitcontrol.es y redirigiéndolas internamente a localhost:8085
- Spring Boot corre en el puerto interno **8085** bajo el perfil **tfg**, gestionado como servicio systemd con el usuario del sistema **orbitcontrol**
- MySQL corre localmente en el mismo servidor con la base de datos **orbitcontroltfgbd**
- **SSL** gestionado por **Certbot** con certificados Let's Encrypt, con renovación automática

```
certbot --nginx -d api-tfg.orbitcontrol.es
```

Acceso al servidor por SSH

Para conectarse al servidor es necesaria la clave SSH [~/.ssh/orbitcontrol](#). El acceso se realiza con el siguiente comando:

```
ssh -i ~/.ssh/orbitcontrol root@178.105.30.196
```



Despliegue manual en el entorno TFG

El despliegue completo se realiza con un único comando desde la raíz del proyecto:

```
npm run deploy:tfg
```

Este comando ejecuta internamente tres pasos en secuencia:

```
# 1. Compila el JAR de Spring Boot
./mvnw package -DskipTests

# 2. Sube el JAR al servidor mediante SCP
scp -i ~/.ssh/orbitcontrol \
    target/OrbitControl-0.0.1-SNAPSHOT.jar \
    root@178.105.30.196:/opt/orbitcontrol/app-tfg.jar

# 3. Reinicia el servicio en el servidor
ssh -i ~/.ssh/orbitcontrol root@178.105.30.196 \
    "systemctl restart orbitcontrol-tfg"
```

Para consultar los logs del servidor en tiempo real tras el despliegue:

```
npm run logs:tfg
```

Despliegue automático CI/CD

El despliegue automático se activa con un push a la rama tfg:

```
git push origin tfg
```

GitHub Actions detecta el push, compila el **JAR**, lo sube al servidor mediante **SCP** usando la **clave SSH** almacenada como secret en GitHub, y reinicia el servicio.

En paralelo, Vercel detecta el push y despliega el frontend automáticamente. El proceso completo tarda aproximadamente **3-5 minutos**.

Recuperación del acceso SSH



Si se pierde la clave SSH `~/.ssh/orbitcontrol` o se pierde el acceso al servidor, existe una forma de recuperarla.

Accede al servidor desde el panel de Hetzner sin necesidad de SSH:

1. Entra en console.hetzner.cloud
2. Selecciona el servidor correspondiente
3. Pulsa **Console** en la barra superior, se abre una terminal directa en el navegador
4. Inicia sesión con `root` y la contraseña del servidor

Una vez dentro, recupera la clave privada que ya existe en el servidor:

```
cat /root/.ssh/orbitcontrol
```

5.10.- Presentación De La Aplicación

La presentación de Orbit Control se ha realizado en formato vídeo con una duración aproximada de 5 minutos.

El vídeo muestra el funcionamiento completo de la aplicación recorriendo los módulos principales: autenticación mediante los tres métodos disponibles, dashboard, gestión del obrador con una elaboración completa, almacén con alertas de stock, creación y seguimiento de un pedido, y gestión de usuarios.

El vídeo está disponible en el siguiente enlace:

<https://youtu.be/9rCR-WdBKU>

6.- Planificación Del Proyecto

La planificación de Orbit Control se organizó en nueve fases que van desde el análisis inicial del sector hasta la entrega de la documentación.

No fue una planificación rígida desde el principio, sino que fue evolucionando a medida que avanzaba el desarrollo y aparecían nuevas necesidades que no estaban previstas al principio.

6.1 Análisis y estudio de necesidades (octubre 2025)

Todo empezó por conocer el sector de primera mano. Al trabajar en una heladería artesanal, tenía claro cuáles eran los problemas del día a día: hojas de cálculo para controlar el stock, pedidos por WhatsApp sin ningún seguimiento, y cero trazabilidad de lo que se elaboraba en el obrador.

Esta fase consistió en plasmar todo eso por escrito, analizar qué herramientas existían en el mercado y justificar por qué ninguna cubría realmente las necesidades del sector.

6.2 Definición y alcance del proyecto (octubre 2025)

Una vez claro el problema, toca definir la solución. En esta fase decidí qué iba a incluir el proyecto y qué no, porque si no se acota desde el principio, el proyecto se dispara.

Definí los módulos principales, los roles de usuario y los casos de uso más importantes.

También fue el momento de decidir el stack tecnológico: Angular para el frontend, Spring Boot para el backend y MySQL para la base de datos.

6.3 Diseño de la base de datos (octubre – noviembre 2025)

Esta fue una de las fases más importantes y también de las más complicadas.

Diseñar bien la base de datos desde el principio es clave porque luego cambiarlo todo es muy costoso.

Estuve dándole muchas vueltas a cómo modelar correctamente la relación entre los helados del catálogo y las unidades físicas elaboradas en el obrador, que al final derivó en la separación entre **helados** y **helados_elaborados**.

También tuve que pensar cómo gestionar los lotes de forma genérica sin tener que crear una tabla por cada tipo de entidad, lo que me llevó a usar una relación polimórfica con los campos **tipo_entidad** y **entidad_id**.

6.4 Planificación técnica y arquitectura (*noviembre 2025 – enero 2026*)

Con el diseño de la base de datos listo, tocó montar la base del proyecto.

Se configuró el repositorio de GitHub con las ramas de trabajo, se creó la estructura de carpetas del backend y el frontend, y se configuraron los entornos de desarrollo.

También fue en esta fase donde se empezó a definir la arquitectura en capas del backend y la organización por módulos del frontend.

6.5 Desarrollo backend (*febrero – marzo 2026*)



Empecé por el backend porque sin **API** no hay nada que mostrar en el frontend.

Fui construyendo módulo a módulo: primero la autenticación con JWT y los tres métodos de login, luego los módulos de helados y almacén que eran los más complejos, y después pedidos, proveedores y usuarios.

Lo más complejo fue implementar la lógica de negocio de la elaboración, que al confirmar tiene que crear los registros en **helados_elaborados**, actualizar el stock del helado y descontar los ingredientes del almacén según la receta, todo en la misma transacción.

6.6 Desarrollo frontend (marzo – abril 2026)

Con la API funcionando empecé a construir el frontend en Angular.

Fui módulo a módulo conectando cada vista con sus endpoints correspondientes.

Lo que más tiempo me llevó fue el módulo de pedidos, porque tiene muchos estados, las dos pestañas de pendientes, el modal de recepción con los lotes, y el borrador automático.

También le dediqué bastante tiempo al diseño responsive para que todo se viera bien tanto en ordenador como en tablet y móvil.

6.7 Infraestructura y CI/CD (abril – mayo 2026)



Esta fue la fase donde más aprendí de todo el proyecto, y gran parte de ese aprendizaje vino directamente de mis prácticas en **Airzone**.

Allí fue la primera vez que vi cómo funciona un **pipeline de CI/CD** real, cómo se separan los entornos de desarrollo y producción, y cómo el código pasa de una rama de GitHub a estar desplegado en un servidor sin tocar nada a mano.

Me basé en lo que había visto en **Airzone** para configurar el servidor Hetzner: instalé MySQL, configuré Nginx como reverse proxy para que las peticiones llegaran al backend sin exponer el puerto directamente, generé el certificado **SSL** con **Certbot** para tener HTTPS y monté el servicio **systemd** para que el backend arranque automáticamente cuando el servidor se reinicia.

Para el **CI/CD** configuré **GitHub Actions** igual que lo había visto en Airzone: un **workflow** que se dispara con un push a la rama correspondiente, **compila** el **JAR** con Maven, lo sube al servidor por **SCP** y **reinicia** el servicio.

Para el frontend aproveché la integración nativa de Vercel con GitHub, que detecta el push y **despliega solo**.

El resultado es que con un **git push** el entorno está actualizado en **menos de 5 minutos sin intervenir manualmente**.

6.8 Pruebas y ajustes finales (**mayo 2026**)

Con todo desplegado, estuve probando todos los flujos de principio a fin: crear pedidos, registrar elaboraciones, consumir stock, gestionar usuarios...

Fui corrigiendo los errores que iban apareciendo y ajustando detalles de la interfaz que no me convencían del todo.

6.9 Documentación TFG (*mayo 2026*)

La última fase fue escribir esta documentación, recogiendo todo el proceso de desarrollo desde el análisis hasta los manuales.

Fue más trabajo del que esperaba, pero también sirvió para ordenar y reflexionar sobre todas las decisiones que fui tomando a lo largo del proyecto.

7.- Control De La Ejecución

A lo largo del desarrollo se fueron aplicando una serie de mecanismos para asegurar que el proyecto avanzaba correctamente y que el código funcionaba según lo esperado.

7.1 Evaluación de las actividades realizadas

El control del avance se hizo de forma continua comparando lo que se iba desarrollando con los requisitos definidos al inicio del proyecto.

Al terminar cada módulo verificaba que cubría todos los casos de uso definidos en la fase de análisis: que los roles tenían los permisos correctos, que los flujos principales funcionaban de principio a fin y que los casos de error estaban contemplados y gestionados correctamente.

Para el seguimiento del código se utilizó **GitHub** con ramas diferenciadas. La rama *dev* se usó para el desarrollo del día a día, y los cambios solo pasaban a *main* cuando estaban probados y estables.

Esto permitió tener siempre una versión funcional desplegada sin que los cambios en desarrollo la afectaran.

7.2 Indicadores de calidad

Los principales indicadores que se fueron revisando a lo largo del desarrollo fueron los siguientes:

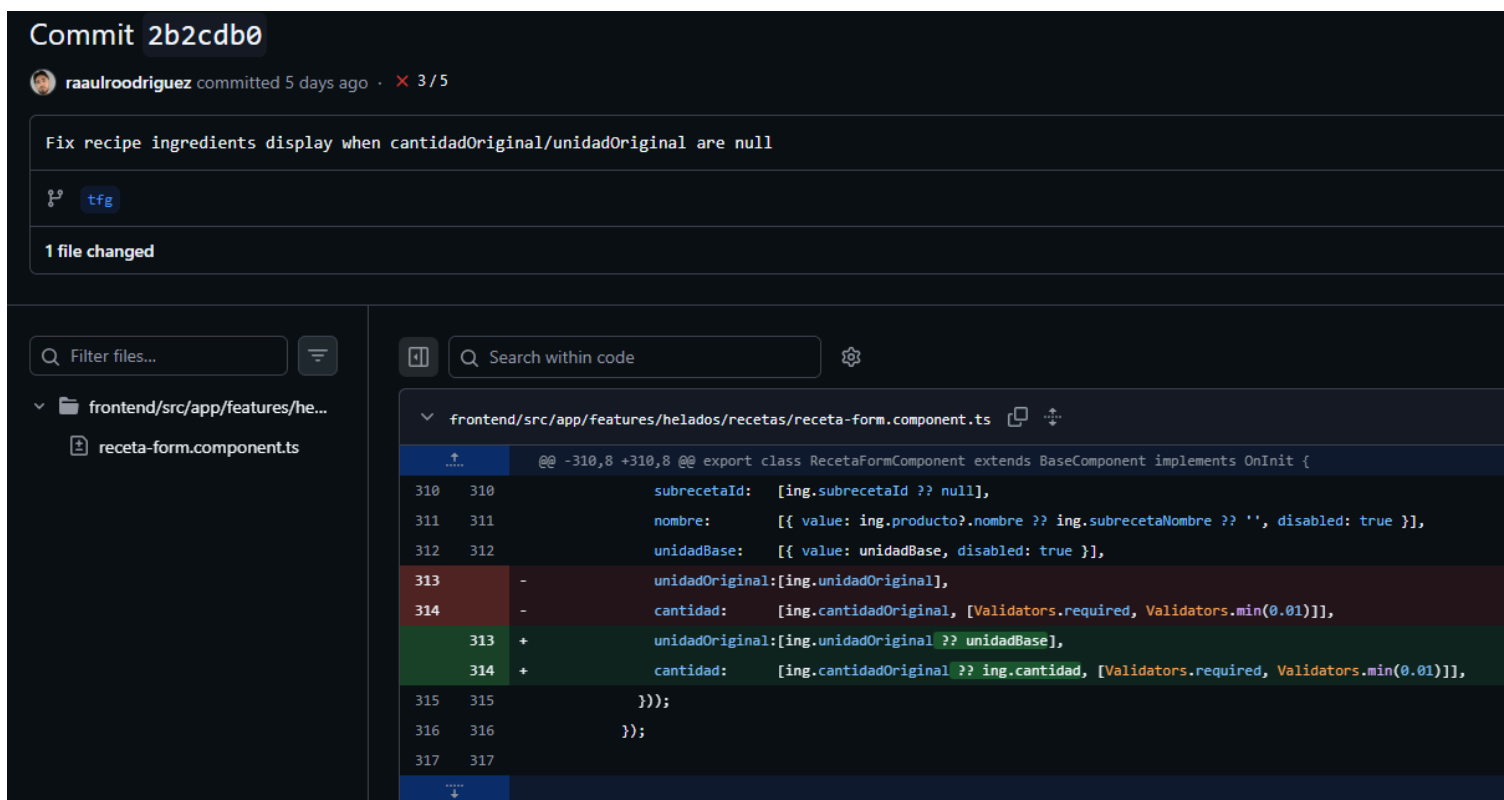
- **Cobertura funcional:** todos los módulos planificados en la fase de definición quedaron implementados en la versión final.
- **Consistencia de la interfaz:** todos los módulos siguen el mismo patrón visual y de interacción, con los mismos componentes compartidos de **shared/**, los mismos colores y la misma estructura de navegación.
- **Validación de formularios:** todos los formularios de la aplicación validan los datos antes de permitir el envío, con mensajes de error claros y el botón de acción deshabilitado hasta que los campos obligatorios están correctamente rellenos.
- **Control de acceso por roles:** se verificó que cada ruta y cada endpoint respondía correctamente según el rol del usuario autenticado, devolviendo un **error 403** cuando se intenta acceder sin los permisos necesarios.
- **Responsive:** se comprobó el funcionamiento de la aplicación en diferentes tamaños de pantalla, verificando que las tablas se transforman correctamente en cards en móvil y que la interfaz es usable desde cualquier dispositivo.
- **Testing de la API con Postman:** durante el desarrollo del backend se utilizó Postman para probar cada **endpoint** antes de conectarlo con el frontend. Esto permitió detectar y corregir problemas en la API de forma aislada, sin depender de que el frontend estuviera terminado.

7.3 Registro y evaluación de incidencias

Durante el desarrollo fueron apareciendo varios problemas que hubo que identificar, analizar y resolver. Los más destacados se detallan en el apartado 8 de este documento.

El procedimiento seguido en cada caso fue el mismo: identificar el problema, entender la causa raíz, buscar la solución más limpia y verificar que no afectaba a otras partes del sistema antes de darlo por resuelto.

Para el registro de incidencias se usó el propio sistema de **commits** de GitHub, donde cada **fix** quedaba documentado con un mensaje descriptivo indicando qué se había corregido.



The screenshot shows a GitHub commit interface for commit `2b2cdb0` by user `raulrodriguez`. The commit message is "Fix recipe ingredients display when cantidadOriginal/unidadOriginal are null". Below the message, it indicates "1 file changed". The code diff shows changes in `frontend/src/app/features/helados/recetas/receta-form.component.ts`. The diff highlights the following changes:

```
@@ -310,8 +310,8 @@ export class RecetaFormComponent extends BaseComponent implements OnInit {
 310 310     subrecetaId: [ing.subrecetaId ?? null],
 311 311     nombre:      [{ value: ing.producto?.nombre ?? ing.subrecetaNombre ?? '', disabled: true }],
 312 312     unidadBase:  [{ value: unidadBase, disabled: true }],
 313 -    unidadOriginal:[ing.unidadOriginal],
 314 -    cantidad:      [ing.cantidadOriginal, [Validators.required, Validators.min(0.01)]],
 313 +    unidadOriginal:[ing.unidadOriginal ?? unidadBase],
 314 +    cantidad:      [ing.cantidadOriginal ?? ing.cantidad, [Validators.required, Validators.min(0.01)]],
 315 315     });
 316 316     });
 317 317 }
```

7.4 Participación de usuarios en la evaluación

Durante el desarrollo se contó con la colaboración de compañeros de trabajo de la heladería como usuarios de **prueba reales**.

Se les pidió que probaran los **flujos principales** de la aplicación, especialmente los del obrador y el almacén, que son los que más usan en su día a día.

Su **feedback** fue útil para detectar problemas de usabilidad que no se aprecian cuando eres el propio desarrollador, como la necesidad de confirmar ciertas acciones antes de ejecutarlas o mejorar algunos mensajes de error para que fueran más claros.

7.5 Cumplimiento de las especificaciones

Al finalizar el desarrollo se verificó que la aplicación cumplía con todas las especificaciones definidas al inicio del proyecto.

Los siete módulos planificados quedaron implementados, el sistema de autenticación con los tres métodos funciona correctamente, la aplicación es accesible desde cualquier dispositivo como **PWA**, y el despliegue está completamente **automatizado** mediante **CI/CD**.

8.- Dificultades Encontradas

A lo largo del desarrollo fueron apareciendo una serie de dificultades técnicas que obligaron a replantear decisiones, buscar soluciones o aprender conceptos que no dominaba al principio del proyecto.

8.1 Separación entre helados y helados_elaborados

Una de las decisiones de diseño que más me costó fue la separación entre el helado como producto del catálogo y cada unidad física elaborada en el obrador.

Al principio tenía todo en una sola tabla, pero pronto me di cuenta de que eso no permitía tener **trazabilidad real** de cada producción. No podía saber cuándo se había elaborado cada unidad, cuál era su fecha de caducidad o en qué estado estaba.

Tuve que replantear el modelo de datos a mitad del desarrollo y crear la tabla **helados_elaborados**, lo que implicó modificar bastantes partes del backend que ya estaban hechas.

8.2 Relación polimórfica en lotes

La gestión de lotes fue otro quebradero de cabeza. Necesitaba asociar lotes tanto a helados como a productos del almacén, y estuve dándole vueltas a cómo modelarlo bien.

La primera opción que se me ocurrió fue crear una **tabla intermedia** por cada tipo de entidad, es decir, una tabla **lotes_helados** y otra **lotes_productos**. El problema es que eso **duplica la lógica**, y si en el futuro hubiera que añadir otro tipo de entidad habría que crear otra tabla más, lo que no escala bien.

La segunda opción era tener **columnas separadas** en la propia tabla **lotes**, con un campo **id_helado** y otro **id_producto**, dejando **nulo** el que no aplique en cada caso. Esto es más sencillo de entender pero genera **columnas nulas constantemente** y tampoco es una solución limpia.

Al final opté por una **relación polimórfica** con los campos **tipo_entidad** y **entidad_id**. En lugar de referenciar directamente una tabla concreta, se guarda el tipo de entidad como texto (**HELADO** o **PRODUCTO**) y el id de esa entidad. Así con **una sola tabla** se pueden gestionar lotes de cualquier tipo sin duplicar nada.

La estructura final de la tabla **lotes** quedó así:

CAMPO	TIPO	DESCRIPCIÓN
id	bigint	Clave primaria
tipo_entidad	enum	HELADO o PRODUCTO
entidad_id	bigint	ID del helado o producto al que pertenece
cantidad	double	Cantidad del lote
fecha_entrada	datetime	Cuándo entró el lote
fecha_caducidad	datetime	Caducidad del lote

Por ejemplo, un registro con **tipo_entidad = HELADO** y **entidad_id = 3** pertenece al helado con **id 3**, y otro con **tipo_entidad = PRODUCTO** y **entidad_id = 7** pertenece al producto con **id 7**.

Fue la primera vez que implementaba algo así y costó bastante entenderlo, pero es la solución más limpia y la que mejor escala si en el futuro hubiera que añadir nuevos tipos de entidad.

8.3 Sistema de roles y permisos

Controlar los permisos en dos sitios a la vez, frontend y backend, fue más complicado de lo que parecía al principio.

En el backend hay que configurar **Spring Security** para que cada **endpoint** solo sea accesible por los roles correspondientes, y en el frontend hay que proteger las rutas con guards de Angular y ocultar o deshabilitar los elementos de la interfaz según el rol del usuario conectado.

Lo más difícil fue asegurarse de que ambas capas estuvieran **sincronizadas**, porque si proteges un endpoint en el backend pero olvidas ocultar el botón en el frontend el usuario ve una opción que luego le da error.

8.4 DTOs y MapStruct

Al principio no entendía muy bien para qué servían los **DTOs** ni por qué no devolver directamente las entidades **JPA**.

El primer problema que me encontré fueron las referencias circulares. Cuando una entidad tiene una relación bidireccional con otra, por ejemplo un **Pedido** que tiene una lista de **ItemPedido** y cada **ItemPedido** tiene una referencia de vuelta al **Pedido**, al intentar devolver esa entidad entra en un bucle infinito.

La solución más rápida es anotar uno de los lados de la relación con **@JsonIgnore** para que Jackson no lo serialice, pero eso tiene un problema: estás modificando la entidad para adaptarla a la API.

La solución limpia es usar **DTOs**, donde controlas exactamente qué campos incluyes en cada respuesta sin tocar el modelo de base de datos.

Una vez entendido el concepto, el siguiente problema fue **MapStruct**.

Genera el código de mapeo automáticamente, pero cuando hay relaciones anidadas hay que configurar explícitamente cómo mapear cada nivel. Si no lo haces bien o te falta información en la respuesta o te vuelve a dar el mismo error de bucle.

Me costó un tiempo hasta que le cogí el truco, pero una vez configurado correctamente ahorra muchísimo código repetitivo.

8.5 CORS

El primer día que intenté conectar el frontend con el backend me encontré con errores de **CORS** en el navegador.

Las peticiones llegaban al servidor pero el navegador las **bloqueaba** porque el frontend estaba en un **origen distinto** al backend.

Tuve que aprender cómo funciona **CORS**, configurar **Spring Security** para permitir las peticiones del frontend y entender por qué en local funciona de una forma y en producción de otra.

Es algo que parece una tontería pero la primera vez que te lo encuentras te puede bloquear un buen rato.

8.6 JWT y Spring Security

Configurar **Spring Security** con **JWT** fue una de las partes **más técnicas** del proyecto.

Spring Security tiene una **cadena de filtros** bastante compleja y al principio me costó entender en qué punto del proceso había que interceptar la petición para validar el token, extraer el usuario y meterlo en el contexto de seguridad.

Además, tener tres métodos de autenticación distintos (contraseña, PIN y RFID) añadía complejidad porque cada uno tiene su propio flujo pero todos tienen que acabar generando el mismo tipo de **token JWT**.

8.7 Responsive y adaptación a móvil

Hacer que la aplicación funcionara bien tanto en ordenador como en tablet y móvil fue más trabajo del esperado.

Las tablas con muchas columnas no caben en pantallas pequeñas, así que tuve que diseñar una versión alternativa en formato card para móvil que mostrará la misma información de forma diferente.

Tailwind CSS ayuda mucho con el **responsive**, pero igualmente hay que pensar bien cada vista para que tenga sentido en los dos formatos.

8.8 Borrador automático de pedidos

Implementar el **guardado automático** del borrador de pedido cuando el usuario **abandona el formulario sin confirmar** fue más complicado de lo que parecía.

Había que detectar cuándo el usuario **navegaba fuera de la página**, guardar el estado del formulario en ese momento y recuperarlo la próxima vez que entrara a crear un pedido.

En Angular esto se gestiona con **guards de salida** y el **ciclo de vida** de los componentes, y tuve que aprender cómo funcionaba todo eso para implementarlo correctamente.

8.9 Autenticación NFC

Integrar el **lector de tarjetas NFC** con la aplicación web fue uno de los retos más interesantes del proyecto.

El lector funciona como un teclado que envía el **UID de la tarjeta** como si fuera texto tecleado y pulsa la tecla **"enter" automáticamente**, así que tuve que **capturar ese input** en el navegador de forma que no interfiriera con el resto de la interfaz.

Además, la pantalla de espera tenía que estar escuchando constantemente sin bloquear el hilo principal, lo que me obligó a aprender cómo gestionar bien los **eventos del teclado** en Angular.

8.10 Variables de entorno y configuración por entornos

Gestionar configuraciones distintas para **local**, **desarrollo** y **producción** fue algo que no había hecho antes y que aprendí en las prácticas en Airzone.

En **Spring Boot** se resuelve con perfiles de configuración (**application-dev.properties**, **application-tfg.properties**), pero al principio no tenía claro cómo funcionaba y metía las credenciales directamente en el código, lo que es una muy mala práctica.

Tuve que refactorizar esa parte para separar bien la configuración sensible del código fuente.

9.- Propuestas De Mejora

Orbit Control es una **aplicación funcional** y **operativa** en su versión actual, pero durante el desarrollo fueron surgiendo ideas y funcionalidades que por tiempo o complejidad no llegaron a implementarse.

A continuación se describen las propuestas de mejora más relevantes para versiones futuras.

9.1 Exportación de informes en PDF y Excel

Actualmente la consulta de datos se realiza únicamente a través de los listados en pantalla. Una mejora natural sería permitir **exportar** esos listados en formato PDF o Excel directamente desde la aplicación.

Esto sería especialmente útil para el historial de pedidos, el inventario del almacén o el registro de elaboraciones del obrador, ya que permitiría llevar un control documental de la actividad del negocio sin depender de capturas de pantalla o copias manuales.

9.2 Control de jornadas

El módulo de control de jornadas fue una de las funcionalidades que se empezó a diseñar durante el desarrollo pero que tuvo que aplazarse por su complejidad.

La idea era permitir que los empleados **ficharan** la **entrada** y **salida** directamente desde la aplicación, aprovechando el sistema de **autenticación NFC** que ya existe, de forma que acercar la tarjeta al lector no solo iniciara sesión sino que también registrara el fichaje.

El reto principal estaba en gestionar bien los turnos, las horas extras, los descansos y los cierres de jornada automáticos cuando el empleado se olvida de fichar la salida.

Es una funcionalidad que encaja perfectamente con el sistema actual y que aportaría mucho valor a las heladerías con varios empleados.

9.3 Chatbot para consultar información del negocio

Una de las propuestas más interesantes para versiones futuras es integrar un **chatbot** dentro de la propia aplicación que permita al usuario hacer preguntas en **lenguaje natural** sobre el estado del negocio.

Por ejemplo: "¿Cuántos kilos de azúcar me quedan?", "¿Cuál es el proveedor al que más le compro?" o "¿Qué helados están por debajo del mínimo?".

Técnicamente se implementaría conectando la **API** de un **modelo de lenguaje** como Claude o Gemini con acceso a los datos de la base de datos, de forma que las respuestas sean siempre sobre los datos reales del negocio y no genéricas.

Esto convertiría el dashboard en algo mucho **más interactivo** y **reduciría la curva de aprendizaje** para usuarios menos técnicos.

9.4 Auditoría de cambios

Durante el desarrollo se planteó en varias ocasiones la necesidad de registrar automáticamente quién había modificado qué y cuándo en cada entidad.

En un sistema de gestión real esto es casi imprescindible: si alguien borra un producto, modifica el stock o cambia el precio de un pedido, tiene que quedar registrado.

La implementación más limpia en Spring Boot sería usar **Spring Data JPA Auditing**, que permite anotar los campos **creadoPor**, **creadoEn**, **modificadoPor** y **modificadoEn** directamente en las entidades y los rellena automáticamente en cada operación.

Se aplazó por tiempo pero es una de las mejoras más importantes para una versión de producción real.

10.- Conclusión Final

Orbit Control empezó siendo mi proyecto de fin de grado de DAW y ha acabado siendo algo bastante más grande que eso.

Desde el principio el objetivo no era hacer una aplicación para aprobar, sino construir un **producto real** que pudiera llegar al mercado y resolver un problema que conozco de primera mano por trabajar en una heladería. Eso ha marcado toda la forma de enfocar el desarrollo, desde las decisiones de diseño hasta la infraestructura de despliegue.

El resultado es una aplicación web **completamente funcional**, desplegada en un **servidor real**, con un **dominio propio**, **CI/CD automatizado** y **siete módulos operativos** que cubren los procesos principales de una heladería artesanal. No es un proyecto con datos inventados, sino una herramienta que podría empezar a usarse en un negocio real desde el primer día.

A nivel técnico, este proyecto me ha enseñado cosas que no aparecen en los apuntes del ciclo.

Aprender a separar entornos de desarrollo y producción, configurar **pipelines** de **CI/CD** con **GitHub Actions**, gestionar **despliegues automáticos** en un servidor real o construir **componentes reutilizables** que se comparten entre módulos son cosas que no sabía hacer antes de empezar y que ahora forman parte de mi forma natural de trabajar.

Gran parte de ese aprendizaje vino de las prácticas en **Airzone**, donde pude ver cómo se trabaja en un **entorno profesional real** y aplicar esas mismas prácticas en mi proyecto.



Si tuviera que empezar de nuevo, cambiaría la forma de trabajar más que la tecnología.

Fui demasiado directo al código, especialmente al principio, escribiendo funcionalidad tras funcionalidad sin parar a documentar ni a escribir tests.

El resultado es que hay partes del proyecto que funcionan bien pero que son **difíciles de mantener** porque no están bien documentadas, y que cualquier cambio hay que probarlo manualmente porque no hay tests que te avisen si algo se rompe.

En una próxima versión lo primero que haría sería establecer un orden, y realizar tests y especificaciones desde el día uno, antes de escribir una sola línea de funcionalidad.

Las propuestas de mejora recogidas en el apartado anterior, el control de jornadas, el chatbot, la auditoría de cambios y la exportación de informes, no son ideas abstractas sino **funcionalidades concretas** que ya tengo pensadas y que quiero implementar.

El objetivo es que Orbit Control sea un producto real en el mercado.

Orbit Control demuestra que con las herramientas adecuadas, una necesidad real y las ganas de hacerlo bien, un estudiante de DAW puede construir algo que va mucho más allá de un **proyecto académico**.

Orbit Control - Raúl Rodríguez Aponte